



# LAU

Lebanese American University

*Chartered in the State of New York*

[www.lau.edu.lb](http://www.lau.edu.lb)

## SOE

School of Engineering

*Department of Electrical &*

*Computer Engineering*

[www.lau.edu.lb/ece](http://www.lau.edu.lb/ece)



### **BYBLOS CAMPUS**

P.O. Box: 36  
Byblos, Lebanon  
Tel: +961 9 547 262  
Fax: +961 9 546 262

### **BEIRUT CAMPUS**

P.O. Box: 13-5053 Chouran  
Beirut 1102 2801, Lebanon  
Tel: + 961 1 786 456  
Fax: +961 1 867 098

### **NEW YORK HEADQUARTERS**

211 E. 46th Street  
New York, NY 10017-2935, USA  
Tel: +1 212 203 4333  
Fax: +1 212 784 6597

# Intelligent Data Processing and Applications

COE 543/743

## **Chapter 5 - Structural Similarity**

Joe M. TEKLI, PhD

Computer Software Engineering

## Ch. 5 - Structural Similarity

---

### Chapter Contents

1. Introduction
2. String Edit Distance
  - Basic Concepts
  - Main algorithm (Wagner and Fisher 1974)
3. Tree Edit Distance (TED)
  - Basic Approach (Chawathe, 1999)
  - Sub-tree Similarity Approach (Niermand and Jagadish, 2002)
4. Applications of TED-based Similarity
  - Data Warehousing and Version Control
  - Document Classification and Clustering
  - Data Integration
  - Ranked Approximate Querying

## Ch. 5 - Structural Similarity

---

### 1. Introduction

- **Dynamic programming:** is a method for solving complex problems in an **iterative** or **recursive** manner, by breaking them down into **simpler sub-problems**, such as solving the sub-problems comes down to solving the initial problem
  - Applicable to problems having **overlapping sub-problems** and **optimal sub-structure** properties
    - Overlapping sub-problems: If the main problem can be broken down into sub-problems
    - Optimal sub-structure: The main problem's optimal solution can be constructed efficiently from optimal solutions to its sub-problems
      - Dynamic programming is called **divide and conquer** techniques when the **overlapping problems** are much **smaller than the original** problem
  - Examples:
    - *Dijkstra's OSPF* algorithm: using in network routing and graph-based applications
    - *Fibonacci sequence*: using in mathematical-based simulation problems
    - *Sequence alignment* and *tree alignment* (using edit distance)
    - ...

## Ch. 5 - Structural Similarity

---

### 1. Introduction

- With **dynamic programming**
  - The final solution is guaranteed to be the best solution: **optimal solution**
    - ⇒ Since it takes into account all possibilities/all possible solutions
      - And then identifies the best one
- In contrast with **greedy algorithms**
  - Similar to dynamic programming, yet consist in making the **locally optimal choice** at each stage with the hope of finding the global optimum (which is not guaranteed)
    - Applicable to problems exhibiting the property of **overlapping sub-problems**
      - Examples:
        - *Activity selection problem*: selecting activities that do not clash
        - *Matching pursuit algorithm*: used in signal approximation
        - *Decision tree learning* algorithms
        - ...

## Ch. 5 - Structural Similarity

---

### 1. Introduction

- Making use of the **dynamic programming** techniques for finding the *Edit Distance* between semi-structured document trees
  - *Tree alignment* developed in **Combinatorial Pattern Matching**

Finding the **cheapest sequence of edit operations** that can  
**transform one tree into another**

- Rigorously structured (*data-centric*) data
- Fine-grained
- **Main target:** data structure



What is edit distance? How does it work?  
How is it applied to document trees?

⇒ Let's start by covering the original string edit distance problem...

## Ch. 5 - Structural Similarity

---

### 2. String Edit Distance

- Also known as **string alignment** problem
  - Given two strings as input: **A** = “*Hi there*” and **B**= “*Hithor*”
    - It computes the **cost** of the **cheapest sequence of operations** that can **transform one string into another**
- **Edit script**: a sequence of edit operations on the source string, to produce the destination string: **ES(A, B) = < op<sub>1</sub>, op<sub>2</sub>, ..., op<sub>k</sub> >**
  - Where **op<sub>i</sub>** is an edit operation
- By assigning costs to edit operations, we can compute the total edit script cost:

$$\mathbf{Cost}_{\mathbf{ES(A,B)}} = \sum_{i=1}^{|\mathbf{ES(A,B)}|} \mathbf{Cost}_{\mathbf{Op_i}}$$

- **Objective** of string edit distance: identifying the **minimum Cost<sub>ES(A,B)</sub>**

## Ch. 5 - Structural Similarity

---

### 2. String Edit Distance

#### 2.1. Wagner & Fisher Approach

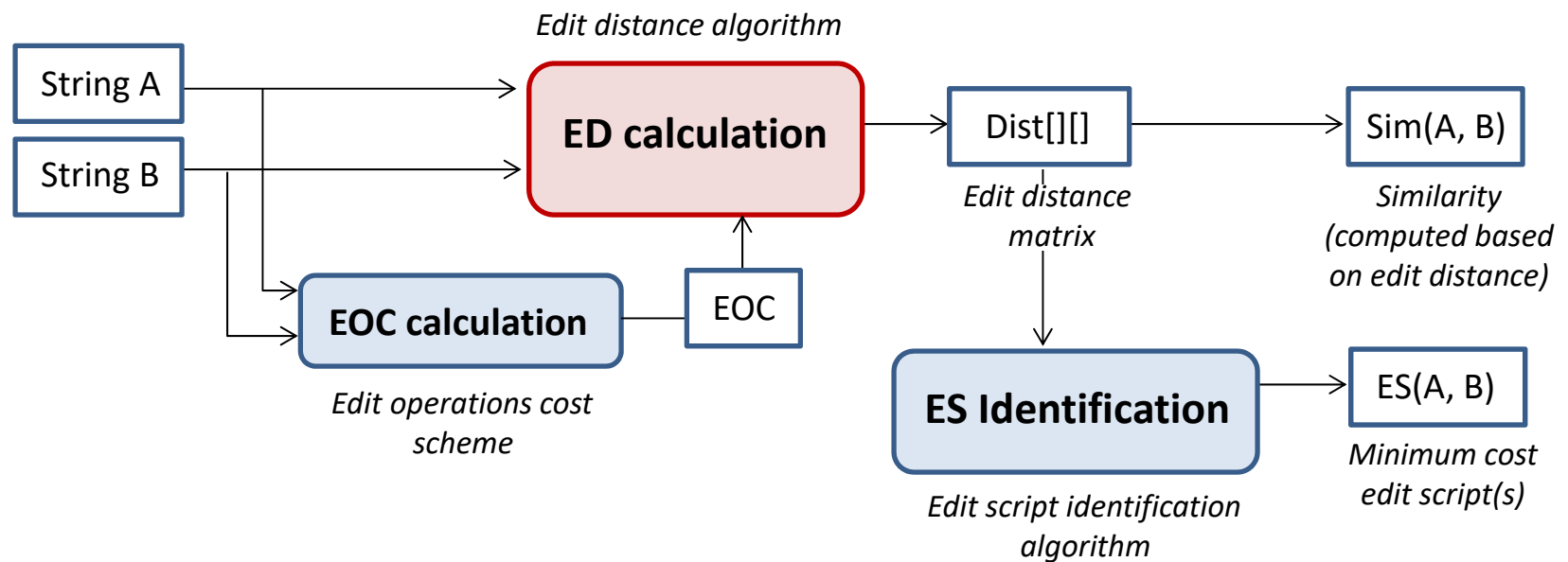
- It is the cornerstone in string edit distance computations
  - Developed by R. A. Wagner and M. J. Fisher in 1974
  - Simple algorithm using **three basic edit operations**:
    - Character **insertion**, noted: **Ins(c)** where **c** is a character
    - Character **deletion**, noted: **Del(c)**
    - Character **update**: noted **Upd(c<sub>1</sub>, c<sub>2</sub>)** where **c<sub>1</sub>** and **c<sub>2</sub>** are characters
  - It takes as **input**: two strings of characters **A** and **B**
  - It produces as **output**: a string edit distance value **Dist(A, B)**
    - NOTE: We can also identify the minimum cost edit script(s): **ES(A, B)**
      - Using a dedicated post-processing algorithm

## Ch. 5 - Structural Similarity

### 2. String Edit Distance

#### 2.1. Wagner & Fisher Approach

- Overall activity diagram:



## Ch. 5 - Structural Similarity

```

Input: A, B           // Input strings
Output: ED (A, B)     // Edit distance between A and B

Begin
Dist [][] = new [0...|A|][0...|B|]           1
Dist[0][0] = 0                               2
For (i = 1 ; i ≤ |A| ; i++) { Dist[i][0] = Dist[i-1][0] + CostDel(Ai) }           3
For (j = 1 ; j ≤ |B| ; j++) { Dist[0][j] = Dist[0][j-1] + CostIns(Bj) }           4
For (i = 1 ; i ≤ |A| ; i++)                   5
{                                               6
    For (j = 1 ; j ≤ |B| ; j++)               7
    {                                           8
        Dist[i][j] = min{                     9
            Dist[i-1][j-1] + CostUpd(Ai, Bj), 10
            Dist[i-1][j] + CostDel(Ai),        11
            Dist[i][j-1] + CostIns(Bj)         12
        }                                     13
    }                                         14
}                                             15
Return Dist[|A|][|B|]                        // Distance between string A and B 16
End

```

W&F's main edit distance algorithm, 1974

## Ch. 5 - Structural Similarity

### 2. String Edit Distance

#### 2.1. Wagner & Fisher Approach

- Example: Consider input strings **A** = "*Hi there*" and **B** = "*Hithor*"

- Having  $|A| = 8$  and  $|B| = 6$ ,  
we draw the following

**edit distance matrix**

- **Rows** designate the costs  
of **deleting** characters  
from the source string **A**

- **Columns** designate the  
costs of **inserting** characters  
into the destination string **B**

- **Diagonals** represent the costs  
of **updating** characters from **A**  
into characters from **B**

|        |     | B → | 'H' | 'i' | 't' | 'h' | 'o' | 'r' |
|--------|-----|-----|-----|-----|-----|-----|-----|-----|
| A<br>↓ |     | 0   | 1   | 2   | 3   | 4   | 5   | 6   |
|        | 0   |     |     |     |     |     |     |     |
|        | 'H' | 1   |     |     |     |     |     |     |
|        | 'i' | 2   |     |     |     |     |     |     |
|        |     | 3   |     |     |     |     |     |     |
|        | 't' | 4   |     |     |     |     |     |     |
|        | 'h' | 5   |     |     |     |     |     |     |
|        | 'e' | 6   |     |     |     |     |     |     |
|        | 'r' | 7   |     |     |     |     |     |     |
|        | 'e' | 8   |     |     |     |     |     |     |

## Ch. 5 - Structural Similarity

---

### 2. String Edit Distance

#### 2.2. Edit Operations

- **Formal** edit operations **syntax**:
  - **Character insertion**: Let  $c$  be a character to be inserted into a string  $S = \langle S_1, S_2, \dots, S_m \rangle$ ,  $Ins(c, i)$  is a character insertion operation applied to  $S$ , inserting  $c$  as the  $i$ th character of  $S$ , transforming  $S$  into  $S' = \langle S'_1, S'_2, \dots, S'_{i-1}, c, S'_{i+1}, \dots, S'_{m+1} \rangle$ 
    - Simplified syntax:  $Ins(c)$  will be used in the remainder to simplify presentation
  - **Character deletion**: Let  $c$  be a character in  $S$ ,  $Del(c)$  is a node deletion operation applied on  $S$  that removes  $c$  from  $S$ , producing  $S' = \langle S'_1, S'_2, \dots, S'_{i-1}, S'_{i+1}, \dots, S'_m \rangle$
  - **Character update**: Given a character  $c_1$  in  $S$ , and another character  $c_2$ ,  $Upd(c_1, c_2)$  is a character update operation applied to  $c_1$  resulting in a string  $S'$  which is identical to  $S$  except that in  $S'$ ,  $c_1$  is replaced with  $c_2$ .

## Ch. 5 - Structural Similarity

---

### 2. String Edit Distance

#### 2.3. Edit Operations Costs

- Different edit operation costs schemes can be used
  - **Intuitive cost model:**
    - $\text{Cost}_{\text{Ins}(c)} = \text{Cost}_{\text{Del}(c)} = 1$ 
      - Unit cost
    - $\text{Cost}_{\text{Upd}(c_1, c_2)} = \begin{cases} 1 & \text{if } c_1 \neq c_2 \\ 0 & \text{otherwise} \end{cases}$
  - **NOTE:** Different edit operation cost models can be chosen by the system designer
    - Based on the application domain, given theoretical/experimental foundations
      - For instance: we would consider the cost of updating a lower case letter into its upper case form to cost less than a normal update operation, etc.

## Ch. 5 - Structural Similarity

---

### 2. String Edit Distance

#### 2.3. Edit Operations Costs

- Example: Consider input strings **A = “Hi there”** and **B= “Hithor”**
  - The **maximum** edit script **cost** possible is: **8 + 6 = 14**
    - Cost of deleting all characters from the source string **A**
    - Plus the cost of inserting all characters from the destination string **B**

$$ED_{\text{Max}}(\mathbf{A}, \mathbf{B}) = \text{Max}(\text{Cost}_{\text{ES}(\mathbf{A}, \mathbf{B})}) = \sum_{i=1}^{|\mathbf{A}|} \text{Cost}_{\text{Del}}(\mathbf{A}_i) + \sum_{j=1}^{|\mathbf{B}|} \text{Cost}_{\text{Ins}}(\mathbf{A}_j)$$

- **NOTE**: Following the general concept edit distance, the **maximum** edit script cost could reach **infinity**:  $\infty$ 
  - The cost of applying two inverse operations an infinite number of times
    - Inserting and then deleting the same character
    - Updating in one direction, followed by an update in the opposite direction

## Ch. 5 - Structural Similarity

### 2. String Edit Distance

#### 2.4. Computing Edit Distance

- Example: Consider input strings **A = “Hi there”** and **B= “Hithor”**

- Running the edit distance algorithm populates the edit distance matrix as follows:

- The highlighted cell designates the minimum edit distance

|     |     | B → | 'H' | 'i' | 't' | 'h' | 'o' | 'r' |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| A ↓ | 0   | 0   | 1   | 2   | 3   | 4   | 5   | 6   |
|     | 'H' | 1   | 0   | 1   | 2   | 3   | 4   | 5   |
|     | 'i' | 2   | 1   | 0   | 1   | 2   | 3   | 4   |
|     |     | 3   | 2   | 1   | 1   | 2   | 3   | 4   |
|     | 't' | 4   | 3   | 2   | 1   | 2   | 3   | 4   |
|     | 'h' | 5   | 4   | 3   | 2   | 1   | 2   | 3   |
|     | 'e' | 6   | 5   | 4   | 3   | 2   | 2   | 3   |
|     | 'r' | 7   | 6   | 5   | 4   | 3   | 3   | 2   |
|     | 'e' | 8   | 7   | 6   | 5   | 4   | 4   | 3   |

## Ch. 5 - Structural Similarity

### 2. String Edit Distance

#### 2.4. Computing Edit Distance

- Example: Consider input strings **A** = “*Hi there*” and **B**= “*Hithor*”
  - Given the edit distance matrix in the previous slide
    - The **minimum** edit script **cost** is: 3
$$ED(A, B) = \text{Min}(\text{Cost}_{ES(A, B)}) = \text{Dist}[|A|, |B|] = \text{Dist}[8, 6] = 3$$
  - The cost of edit operations **necessary** to transform string **A** to **B**



How to identify these operations?

⇒ By post-processing the edit distance matrix...



## Ch. 5 - Structural Similarity

### 2. String Edit Distance

#### 2.5. Edit Script Identification

- Example: Consider input strings **A = “Hi there”** and **B= “Hithor”**

- We post-process the populated edit distance matrix in order to identify the **minimum cost edit script(s)** shown here as highlighted cells
- There could be **more than one** minimum cost ES!

|        |     | B → | 'H' | 'i' | 't' | 'h' | 'o' | 'r' |
|--------|-----|-----|-----|-----|-----|-----|-----|-----|
| A<br>↓ | 0   | 0   | 1   | 2   | 3   | 4   | 5   | 6   |
|        | 'H' | 1   | 0   | 1   | 2   | 3   | 4   | 5   |
|        | 'i' | 2   | 1   | 0   | 1   | 2   | 3   | 4   |
|        |     | 3   | 2   | 1   | 1   | 2   | 3   | 4   |
|        | 't' | 4   | 3   | 2   | 1   | 2   | 3   | 4   |
|        | 'h' | 5   | 4   | 3   | 2   | 1   | 2   | 3   |
|        | 'e' | 6   | 5   | 4   | 3   | 2   | 2   | 3   |
|        | 'r' | 7   | 6   | 5   | 4   | 3   | 3   | 2   |
|        | 'e' | 8   | 7   | 6   | 5   | 4   | 4   | 3   |

## Ch. 5 - Structural Similarity

---

### 2. String Edit Distance

#### 2.5. Edit Script Identification

- Example: Consider input strings **A = “Hi there”** and **B= “Hithor”**
  - In this example, we identify only one minimum cost edit script:
    - **Raw edit script**: obtained by navigating the ED matrix from **Dist[|A|, |B|]** to **Dist[0, 0]**

$$ES(A, B) = < Del(A_8), Upd(A_7, B_6), Upd(A_6, B_5), Upd(A_5, B_4), Upd(A_4, B_3), \\ Del(A_3), Upd(A_2, B_2), Upd(A_1, B_1) >$$
    - **Final edit script**: obtained after
      - Eliminating costless (useless) edit operations
      - Inversing the order of edit operations

$$ES(A, B) = < Del(A_3), Upd(A_6, B_5), Del(A_8) >$$

## Ch. 5 - Structural Similarity

---

### 2. String Edit Distance

#### 2.5. Edit Script Identification

- Example: Consider input strings **A = “Hi there”** and **B= “Hithor”**
  - If we compute  $\text{Dist}(B, A)$ , we get the same edit distance value:  **$\text{ED}(B, A) = 3$** 
    - Since the intuitive **edit operations cost model** we adopted is **symmetric**
      - Given that *W&F’s* algorithm produces a symmetric distance
    - The corresponding edit script is also “symmetric”:  
 **$\text{ES}(B, A) = \langle \text{Ins}(A_3), \text{Upd}(B_5, A_6), \text{Ins}(A_8) \rangle$** 
      - ⇒ Using the simplified syntax of the character insertion operation
        - More **formally**:  **$\text{ES}(B, A) = \langle \text{Ins}(A_3, \mathbf{3}), \text{Upd}(B_5, A_6), \text{Ins}(A_8, \mathbf{7}) \rangle$** 
          - ⇒ Including positions at which characters should be inserted in the source string
  - **NOTE**: The *ES Identification* algorithm will not be provided in class
    - Writing the algorithm and coding it is part of the **students’ projects**

## Ch. 5 - Structural Similarity

---

### 2. String Edit Distance

#### 2.6. Similarity Computation

- Different formulas can be used to compute similarity based on edit distance
  - Following the formal definition of similarity as the inverse of a distance function
- Given two input strings A and B
  - Formula 1:  $\text{Sim}(A, B) = \frac{1}{1 + \text{ED}(A, B)} \in ]0, 1]$ 
    - Typically used with unbounded distance functions, i.e.,  $\text{Dist}(A, B) \in [0, \infty[$
  - Formula 2:  $\text{Sim}(A, B) = 1 - \frac{\text{ED}(A, B)}{|A| + |B|} \in [0, 1]$ 
    - Where distance is bounded, i.e.,  $\text{ED}(A, B) \in [0, (|A| + |B|)]$

## Ch. 5 - Structural Similarity

---

### 2. String Edit Distance

#### 2.6. Similarity Computation

- Note that both **similarity functions** (1 and 2) are consistent with the formal definition of similarity, and come down to **metrics**
  - Verifying all metric properties, e.g.: *reflexivity*, *symmetricity*, and *triangular inequality* (Ch. 2)
- Given two input strings **A = “Hi there”** and **B= “Hithor”**
  - Formula 1:  $\text{Sim}(\mathbf{A}, \mathbf{B}) = \frac{1}{1 + \text{ED}(\mathbf{A}, \mathbf{B})} = \frac{1}{1 + 3} = 0.25$
  - Formula 2:  $\text{Sim}(\mathbf{A}, \mathbf{B}) = 1 - \frac{\text{ED}(\mathbf{A}, \mathbf{B})}{|\mathbf{A}| + |\mathbf{B}|} = 1 - \frac{3}{8 + 6} \simeq 0.7857$
- **NOTE**: Comparing similarity functions, to decide which one to use, is covered in Ch. 4

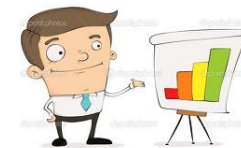
## Ch. 5 - Structural Similarity

---

### 2. String Edit Distance

#### 2.7. Complexity Analysis

- The **time complexity** of  $W\&F$ 's algorithm is  $O(|A| \times |B|)$ 
  - Where  $|A|$  and  $|B|$  designate the cardinalities (in number of characters) of the strings being compared
    - Which amounts to  **$O(N^2)$** 
      - With  $N = \max\{|A|, |B|\}$
- The **space complexity** of  $W\&F$ 's algorithm is also  $O(|A| \times |B|)$ 
  - Which corresponds to the memory size required to store the edit distance matrix
- The algorithm is of **quadratic** time & space complexity with respect to string size
  - It is linearly dependent on the size of each string

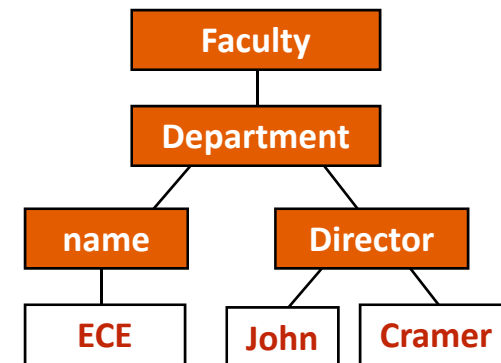


## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

- Semi-structured documents represent hierarchically structured information
  - Can be modeled as **Rooted Ordered Labeled Trees** (ROLTs) following DOM<sup>1</sup>

```
<?XML?>
<Faculty>
  <Department name = "ECE">
    <Director>John Cramer</Director>
  </Department>
</Faculty>
```



- Comparing semi-structured documents comes down to comparing their trees
  - Central solution to compare rigorously structured data: **Tree Edit Distance**

<sup>1</sup> Document Object Model, <http://www.w3.org/DOM>

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.1. Document Tree Representation

- **Rooted Ordered Labeled Tree (ROLT):** It is a rooted tree in which the nodes are labeled and ordered. We denote by  $T[i]$  the  $i^{th}$  node of  $T$  in preorder traversal,  $T[i].\ell$  its label, and  $T[i].d$  its depth.  $R(T)=T[0]$  designates the root node of tree  $T$
- **Semi-structured Document Tree:** It is represented as a ROLT with a node corresponding to each element and attribute
  - Element nodes are labeled with element tag names
  - Attribute nodes appear as children of their encompassing element nodes, labeled and sorted by attribute name, and appearing before all sub-element siblings
  - Element/attribute values can be considered or disregarded following the application at hand
    - Values are disregarded when studying the structural properties of semi-structured documents
      - ⇒ *Structure-only* similarity evaluation

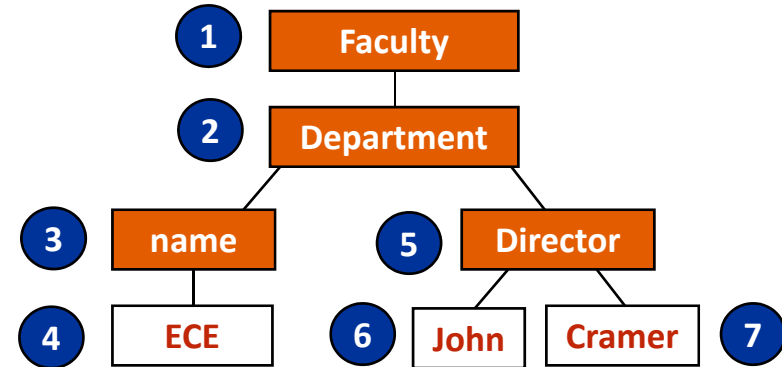
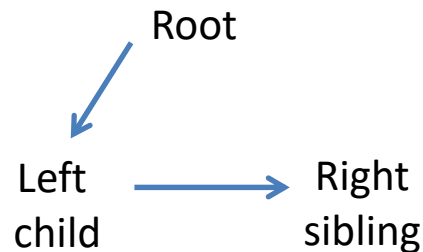
## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.1. Document Tree Representation

- **Tree ordering:** we distinguish three main kinds of tree node traversal

##### 1. Pre-order traversal



↳ Natural order of appearance of elements/attributes in the semi-structured document

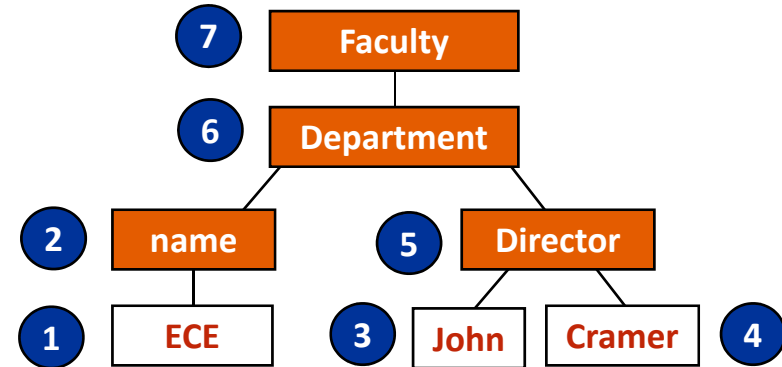
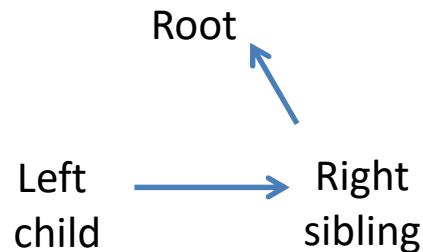
## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.1. Document Tree Representation

- **Tree ordering:** we distinguish three main kinds of tree node traversal

#### 2. Post-order traversal



↳ Traversing document tree from innermost to outermost (root) element

## Ch. 5 - Structural Similarity

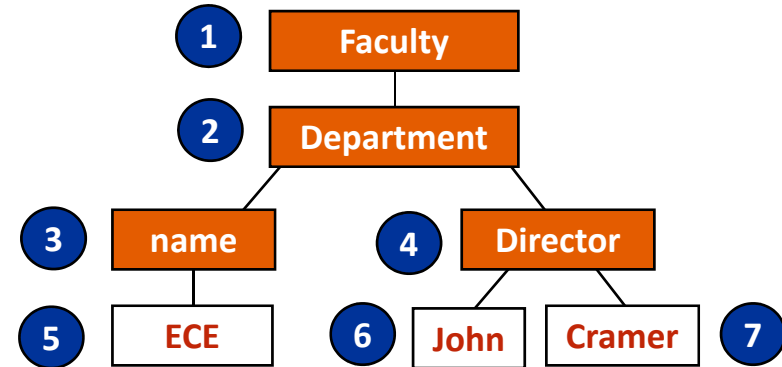
### 3. Tree Edit Distance

#### 3.1. Document Tree Representation

- **Tree ordering:** we distinguish three main kinds of tree node traversal

#### 3. Level-order traversal

Left node  Right node



Traversing document tree left-to-right  
per hierarchical level

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

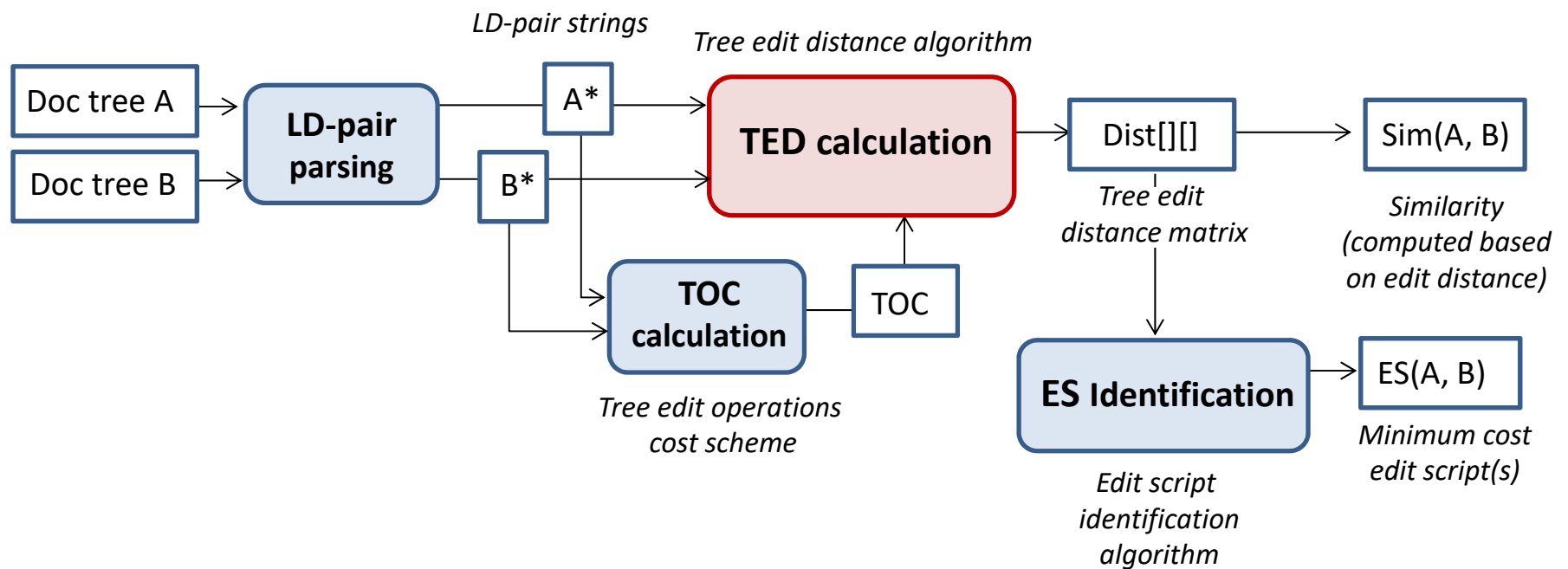
- Basic tree edit distance algorithm
  - Developed by *S.S. Chawathe* in 1999
- Direct application of the famous *W&F algorithm*
  - Which was proven optimal in a wide variety of applications
- **Main procedure:**
  1. Transforming semi-structured documents into *special strings*
    - Made of sequences of special characters
  2. Comparing the strings using an *adaptation of the W&F algorithm*
    - Using dedicated tree edit operations

## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

##### 3.2.1. Overall activity diagram



## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

##### 3.2.2. LD-pair Representation

- Main concept:
  - Transforming a ROLT into a sequence of LD-pair characters
  - Where each character is made of a node label/depth doublet
- **LD-pair doublet**: It is defined, as the pair  $(\ell, d)$  where  $\ell$  and  $d$  are respectively the node's label and depth in the tree. We use  $p.\ell$  and  $p.d$  to refer to the label and the depth of an *ld-pair* node  $p$  respectively
- **LD-pair representation of a tree**: It is the list of the *ld-pair* doublets corresponding to each of the tree nodes, ordered in **pre-order** traversal

## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

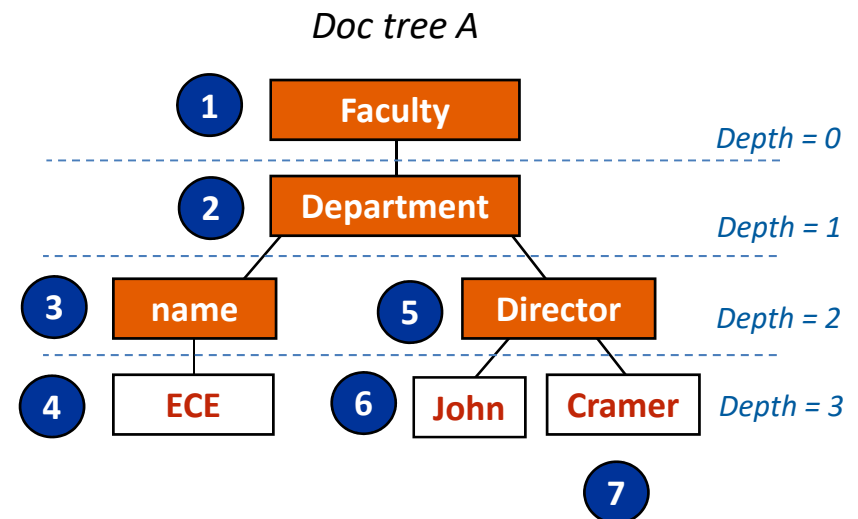
##### 3.2.2. LD-pair Representation

- Example: Considering the sample semi-structured document tree A

LD-pair(A) =

<("Faculty", 0), ("Department", 1),  
 ("name", 2), ("ECE", 3),  
 ("Director", 2), ("John", 3),  
 ("Cramer", 3)>

- We note  $\mathbf{A}^* = \text{LD-pair}(\mathbf{A})$ 
  - Character  $\mathbf{A}_i^*$  in string  $\mathbf{A}^*$  represents node  $\mathbf{A}[i]$  in corresponding tree  $\mathbf{A}$



**Input:** Tree LD-pair representations  $A^*$  and  $B^*$

**Output:** Edit distance between  $A^*$  and  $B^*$ ,  $TED(A^*, B^*)$

Begin

Dist [][] = new [0...|A\*|][0...|B\*|]  
Dist[0][0] = 0

For (i = 1 ; i ≤ |A\*| ; i++) { Dist[i][0] = Dist[i-1][0] + Cost<sub>Del</sub>( $A_i^*$ ) }

For (j = 1 ; j ≤ |B\*| ; j++) { Dist[0][j] = Dist[0][j-1] + Cost<sub>Ins</sub>( $B_j^*$ ) }

For (i = 1 ; i ≤ |A\*| ; i++)

{

For (j = 1 ; j ≤ |B\*| ; j++)

{

Dist[i][j] = min{

If (*Condition*<sub>1</sub> true) { Dist[i-1][j-1] + Cost<sub>Upd</sub>( $A_i^*$ ,  $B_j^*$ ) }

If (*Condition*<sub>2</sub> true) { Dist[i-1][j] + Cost<sub>Del</sub>( $A_i^*$ ) }

If (*Condition*<sub>3</sub> true) { Dist[i][j-1] + Cost<sub>Ins</sub>( $B_j^*$ ) }

}

}

}

Return Dist[|A\*|][|B\*|] // Distance between  $A^*$  and  $B^*$

End

**The Chawathe conditions:**

*Condition*<sub>1</sub> { ( $A[i].d = B[j].d$ ) }

*Condition*<sub>2</sub> { ( ( $A[i].d ≥ B[j].d$ ) or ( $j = |B|$ ) ) }

*Condition*<sub>3</sub> { ( ( $A[i].d ≤ B[j].d$ ) or ( $i = |A|$ ) ) }

Chawathe's main tree edit distance algorithm, 1999

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

##### 3.2.3. Chawathe's TED Conditions

- Condition 1:  $A_i^*.d = B_j^*.d$ 
  - It allows updating one node into another only when the concerned nodes are at the same depth
- Condition 2:  $A_i^*.d \geq B_j^*.d$  or  $j = |B^*|$ 
  - It allows deleting a node from the source tree if its depth is greater or equal to the depth of the corresponding node in the destination tree
    - Intuitively implies that, in order to delete an internal node, all corresponding descendent nodes must be first deleted
- Condition 3:  $A_i^*.d \leq B_j^*.d$  or  $i = |A^*|$ 
  - It allows inserting a node from the destination tree if its depth is greater or equal to the depth of the corresponding node in the source tree
    - Intuitively implies that a node must be inserted before inserting any of its descendants

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

##### 3.2.3. Chawathe's TED Conditions



The conditions do **not** guaranty optimal results!

- Certain tree structural similarities might be overlooked...

The conditions underline **heuristics**

- Producing a result which is close to optimal under certain conditions

**NOTE:** Chawathe's algorithm is **not** heuristic

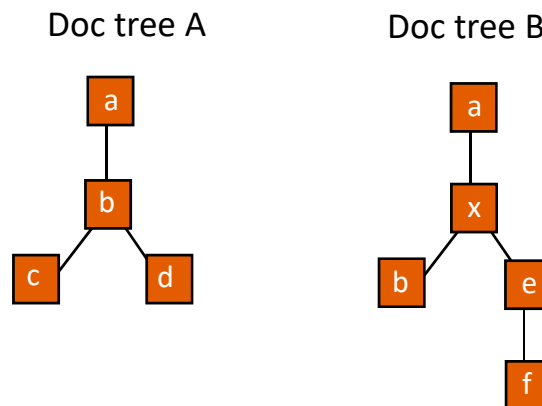
⇒ But the **conditions** are heuristic...

## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

Example: Consider document trees A and B:



- **Step1:** Producing LD-pair representations
  - $\text{LD-pair}(A) = A^* = \langle ("a", 0), ("b", 1), ("c", 2), ("d", 2) \rangle$
  - $\text{LD-pair}(B) = B^* = \langle ("a", 0), ("x", 1), ("b", 2), ("e", 2), ("f", 3) \rangle$

## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

Example: Consider document trees A and B resulting in LD-pair strings  $A^*$  and  $B^*$

- Having  $|A^*| = 4$  and  $|B^*| = 5$ ,  
we draw the following

**edit distance matrix**

- **Rows** designate the costs  
of **deleting** LD-pairs (nodes)  
from the LD-pair representation  
 $A^*$  of the source tree A

- **Columns** designate the costs  
of **inserting** LD-pairs (nodes)  
into the LD-pair representation  
 $B^*$  of the destination tree B

$A^*$   
↓  
(“a”, 0)  
(“b”, 1)  
(“c”, 2)  
(“d”, 2)

| $B^* \rightarrow$ | ("a", 0) | ("x", 1) | ("b", 2) | ("e", 2) | ("f", 3) |   |
|-------------------|----------|----------|----------|----------|----------|---|
|                   | 0        | 1        | 2        | 3        | 4        | 5 |
|                   |          |          |          |          |          |   |
|                   |          |          |          |          |          |   |
|                   |          |          |          |          |          |   |
|                   |          |          |          |          |          |   |
|                   |          |          |          |          |          |   |

- **Diagonals** represent the costs of **updating** LD-pairs (nodes) from  $A^*$  (of tree A) into those in  $B^*$  (of tree B)

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

##### 3.2.4. Tree Edit Operations

- **Formal** edit operations **syntax**:
  - **Leaf node insertion**: Let  $T$  be a ROLT with a node  $p$  having first level sub-trees  $P_1, \dots, P_m$  (i.e., sub-trees rooted at the children nodes of  $p$ ). Given a node  $x$  not belonging to  $T$ ,  $\text{Ins}(x, p, i)$  is a node insertion applied to  $T$ , inserting  $x$  as the  $i$ th child of  $p$ , yielding  $T'$  where  $p$  has first level sub-trees  $P_1, \dots, P_{i-1}, x, P_{i+1}, \dots, P_{m+1}$ 
    - Simplified syntax:  $\text{Ins}(x)$  will be used in the remainder to simplify presentation
  - **Leaf node deletion**: Given a leaf node  $x$  in ROLT  $T$ ,  $x$  having parent node  $p$  with first level sub-trees  $P_1, \dots, P_m$ ,  $\text{Del}(x)$  is a node deletion operation applied on  $T$  that removes  $x$  from  $T$ , yielding  $T'$  where  $p$  has first level sub-trees  $P_1, \dots, P_{i-1}, P_{i+1}, \dots, P_m$
  - **Node update**: Given a node  $x$  in ROLT  $T$ , and a label  $\ell$ ,  $\text{Upd}(x, \ell)$  is a node update operation applied to  $x$  resulting in  $T'$  which is identical to  $T$  except that in  $T'$ ,  $x$  bears  $\ell$  as its label. The update operation could be also formulated as follows:  $\text{Upd}(x, y)$  where  $y$ ,  $\ell$  denotes the new label to be assumed by  $x$

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

##### 3.2.5. Tree Edit Operations Costs

- Different edit operation costs schemes can be used
  - **Intuitive cost model:**
    - $\text{Cost}_{\text{Ins}(n)} = \text{Cost}_{\text{Del}(n)} = 1$ 
      - Unit cost
    - $\text{Cost}_{\text{Upd}(n_1, n_2)} = \begin{cases} 1 & \text{if } n_1 \neq n_2 \\ 0 & \text{otherwise} \end{cases}$
  - **NOTE:** Different edit operation cost models can be chosen by the system designer
    - Based on the application domain, given theoretical/experimental foundations
      - For instance: we would consider the syntactic and/or semantic similarity between node labels and/or values, or node depth, etc.

## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

##### 3.2.6. Computing Tree Edit Distance

Example: Consider document trees A and B resulting in LD-pair strings A\* and B\*

- Running the edit distance algorithm populates the edit distance matrix as follows:

A\*  
↓

- The highlighted cell designates the minimum edit distance

|   |          | B* → ("a", 0) ("x", 1) ("b", 2) ("e", 2) ("f", 3) |   |   |   |   |   |
|---|----------|---------------------------------------------------|---|---|---|---|---|
|   |          | 0                                                 | 1 | 2 | 3 | 4 | 5 |
| 0 | A*       | 0                                                 | 1 | 2 | 3 | 4 | 5 |
| 1 | ("a", 0) | 1                                                 | 0 | 1 | 2 | 3 | 4 |
| 2 | ("b", 1) | 2                                                 | 1 | 1 | 2 | 3 | 4 |
| 3 | ("c", 2) | 3                                                 | 2 | 2 | 2 | 3 | 4 |
| 4 | ("d", 2) | 4                                                 | 3 | 3 | 3 | 3 | 4 |

## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

##### 3.2.6. Computing Tree Edit Distance

Example: Consider document trees A and B resulting in LD-pair strings A\* and B\*

- Given the edit distance matrix in the previous slide
  - The **minimum** edit script **cost** is: 4
$$\text{TED}(A^*, B^*) = \text{Min}(\text{Cost}_{\text{ES}(A^*, B^*)}) = \text{Dist}[|A^*|, |B^*|] = \text{Dist}[4, 5] = 4$$
  - The cost of edit operations **necessary** to transform string A\* to B\*
  - Which represent the edit operations needed to transform trees A to B



How to identify these operations?



## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

##### 3.2.7. Edit Script Identification

Example: Consider document trees A and B resulting in LD-pair strings A\* and B\*

- We post-process the populated edit distance matrix in order to identify the **minimum cost edit script(s)** shown here as highlighted cells
- There could be **more than one** minimum cost ES!

A\*  
↓  
(“a”, 0)  
(“b”, 1)  
(“c”, 2)  
(“d”, 2)

| B* → (“a”, 0) (“x”, 1) (“b”, 2) (“e”, 2) (“f”, 3) |   | 0 | 1 | 2 | 3 | 4 | 5 |
|---------------------------------------------------|---|---|---|---|---|---|---|
| 0                                                 | 0 | 0 | 1 | 2 | 3 | 4 | 5 |
| 1                                                 | 1 | 1 | 0 | 1 | 2 | 3 | 4 |
| 2                                                 | 2 | 2 | 1 | 1 | 2 | 3 | 4 |
| 3                                                 | 3 | 3 | 2 | 2 | 2 | 3 | 4 |
| 4                                                 | 4 | 4 | 3 | 3 | 3 | 3 | 4 |

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

##### 3.2.7. Edit Script Identification

Example: Consider document trees A and B resulting in LD-pair strings  $A^*$  and  $B^*$

– In this example, we identify only one minimum cost edit script:

- **Raw edit script:** navigating the ED matrix from  $\text{Dist}[|A^*|, |B^*|]$  to  $\text{Dist}[0, 0]$

$$ES(A^*, B^*) = \langle \text{Ins}(B^*_5), \text{Upd}(A^*_4, B^*_4), \text{Upd}(A^*_3, B^*_3), \text{Upd}(A^*_2, B^*_2), \text{Upd}(A^*_1, B^*_1) \rangle$$

- **Final edit script:** obtained after i) eliminating costless (useless) edit operations, and ii) inverting the order of edit operations

$$ES(A^*, B^*) = \langle \text{Upd}(A^*_2, B^*_2), \text{Upd}(A^*_3, B^*_3), \text{Upd}(A^*_4, B^*_4), \text{Ins}(B^*_5) \rangle$$

⇒ Using the simplified syntax of the leaf node insertion operation

$$\text{Formally: } ES(A^*, B^*) = \langle \text{Upd}(A^*_2, B^*_2), \text{Upd}(A^*_3, B^*_3), \text{Upd}(A^*_4, B^*_4), \text{Ins}(B^*_5, A^*_4, 1) \rangle$$

⇒ Including parent node and sibling position at which the node should be inserted

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

##### 3.2.7. Edit Script Identification

Example: Consider document trees A and B resulting in LD-pair strings  $A^*$  and  $B^*$

- If we compute  $TED(B^*, A^*)$ , we get the same edit distance value:  $TED(B^*, A^*) = 4$ 
  - Since the intuitive **edit operations cost model** we adopted is **symmetric**
    - Given that the *Chawathe's* algorithm produces a symmetric distance
  - The corresponding edit script is also “symmetric”:  
 $ES(B^*, A^*) = \langle \text{Upd}(B^*_2, A^*_2), \text{Upd}(B^*_3, A^*_3), \text{Upd}(B^*_4, A^*_4), \text{Del}(B^*_5) \rangle$

- **NOTE:** The *ES Identification* algorithm will not be provided in class
  - Writing the algorithm and coding it is part of the **students' projects**

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

##### 3.2.8. Similarity Computation

- Different formulas can be used to compute similarity based on edit distance
  - Following the formal definition of similarity as the inverse of a distance function
- Given two input document trees A and B:
  - Formula 1:  $\text{Sim}(A, B) = \frac{1}{1 + \text{TED}(A, B)} \in ]0, 1]$ 
    - Typically used with unbounded distance functions, i.e.,  $\text{TED}(A, B) \in [0, \infty[$
  - Formula 2:  $\text{Sim}(A, B) = 1 - \frac{\text{TED}(A, B)}{|A| + |B|} \in [0, 1]$ 
    - Where distance is bounded, i.e.,  $\text{TED}(A, B) \in [0, (|A| + |B|)]$

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

##### 3.2.8. Similarity Computation

- Note that following *Chawathe's* approach: **Sim(A, B) = Sim(A\*, B\*)**
  - Which is computed based on TED between LD-pair strings: **TED(A\*, B\*)**
    - Where A\* and B\* are the LD-pair representations of trees A and B respectively
- Given two input document trees considered in our running example:
  - Formula 1:  $\text{Sim}(\mathbf{A}, \mathbf{B}) = \text{Sim}(\mathbf{A}^*, \mathbf{B}^*) = \frac{1}{1 + \text{TED}(\mathbf{A}^*, \mathbf{B}^*)} = \frac{1}{1 + 4} = 0.2$
  - Formula 2:  $\text{Sim}(\mathbf{A}, \mathbf{B}) = \text{Sim}(\mathbf{A}^*, \mathbf{B}^*) = 1 - \frac{\text{TED}(\mathbf{A}^*, \mathbf{B}^*)}{|\mathbf{A}^*| + |\mathbf{B}^*|} = 1 - \frac{4}{4 + 5} \approx 0.5556$
- **NOTE**: Comparing similarity functions, to decide which one to use, is covered in Ch. 4

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

##### 3.2.9. Complexity Analysis

- The **time complexity** of Chawathe's algorithm is  $O(|A| + |B| + |A| \times |B|) = O(|A| \times |B|)$ 
  - Where  $O(|A| + |B|)$  designates the complexity of producing LD-pair strings
  - $|A|$  and  $|B|$  designate the cardinalities (in number of nodes) of the trees being compared, which are equivalent to the cardinalities of LD-pair strings  $|A^*|$  and  $|B^*|$  respectively
    - Hence, complexity simplifies to  **$O(N^2)$** 
      - With  $N = \max\{|A|, |B|\}$
- The **space complexity** of Chawathe's algorithm is also  $O(|A| \times |B|)$ 
  - Which corresponds to the memory size required to store the edit distance matrix
- The algorithm is of **quadratic** complexity with respect to tree size
  - It is linearly dependent on the size of each tree

## Ch. 5 - Structural Similarity

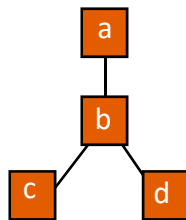
### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

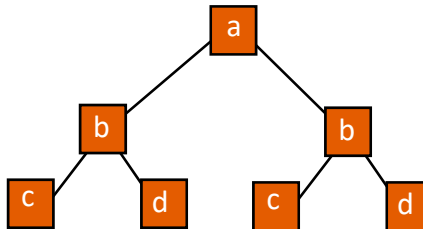
##### 3.2.10. Limitations of Chawathe's approach

- Consider for instance document trees C, D, and E

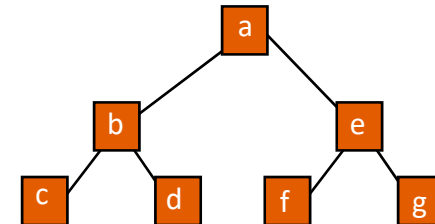
Doc tree C



Doc tree D



Doc tree E



- $TED_{Chawathe}(C^*, D^*) = 3$ , having  $ES(C^*, D^*) = \langle Ins(D^*_5), Ins(D^*_6), Ins(D^*_7) \rangle$
- $TED_{Chawathe}(C^*, E^*) = 3$ , having  $ES(C^*, E^*) = \langle Ins(E^*_5), Ins(E^*_6), Ins(E^*_7) \rangle$

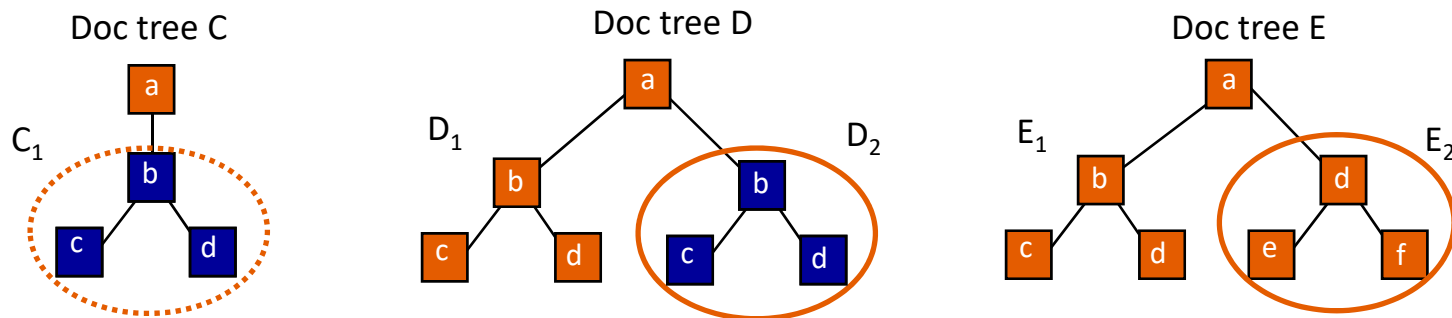
# Ch. 5 - Structural Similarity

## 3. Tree Edit Distance

### 3.2. Chawathe's TED Approach

#### 3.2.10. Limitations of Chawathe's approach

- Consider for instance document trees C, D, and E



$$\text{Sim}_{\text{Chawathe}}(C, D) = 1/1 + \text{TED}_{\text{Chawathe}}(C^*, D^*) = 0.25 = \text{Sim}_{\text{Chawathe}}(C, E)$$

?

Repetition of similar sub-trees **not detected...**

## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

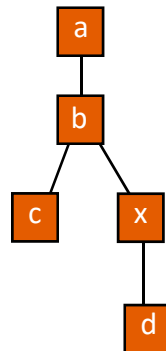
##### 3.2.10. Limitations of Chawathe's approach

- Consider for instance document trees S and T:

Doc tree S



Doc tree T



$TED_{\text{Chawathe}}(S^*, T^*) = 1$   
which does not make sense

?

*It should be equal to 2*

Producing inaccurate results due to  
processing the trees as strings  
using heuristic conditions...

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.2. Chawathe's TED Approach

##### 3.2.10. Limitations of Chawathe's approach

- The reason **sub-tree similarities** go **undetected** and results are sometimes **inaccurate**:
  - Node insertion/deletion operations are executed by **partially considering tree structure**
    - Analyzing adjacent nodes (LD-pairs), and not the tree structure as a whole
      - Using **heuristic conditions**
- Despite the fact that *Chawathe's* algorithm is considered as a reference for modern TED algorithms, and despite that it remains one of the fastest TED approaches (due to its intuitive and straightforward adaptation of the *W&F* algorithm)
  - Yet, it **does not guaranty optimal results**
    - It does not fully process the structural properties of semi-structured document trees
      - By handling them as special strings
        - ⇒ Thus totally **disregarding sub-tree similarities**

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

- TED solution specifically tailored to consider sub-tree similarities
  - Developed by A. Neirman and H.V. Jagadish in 2002
- Motivation: Identifying sub-tree structural similarities is central
  - Frequent presence of **repeated** and **optional** elements in document trees
  - Repeating elements induce multiple occurrences of similar element/attribute sub-trees
    - Presence of optional elements/attributes or identical sub-trees in the same document
      - ⇒ Refer to sample document trees C, D, and F in the previous slide
- Inspired by *Chawathe's* approach, introducing an adapted algorithm:
  - Adding two new edit operations: **insert tree** and **delete tree**
  - Processing trees in their **native structural form**
    - Rather than converting them to special LD-pair strings

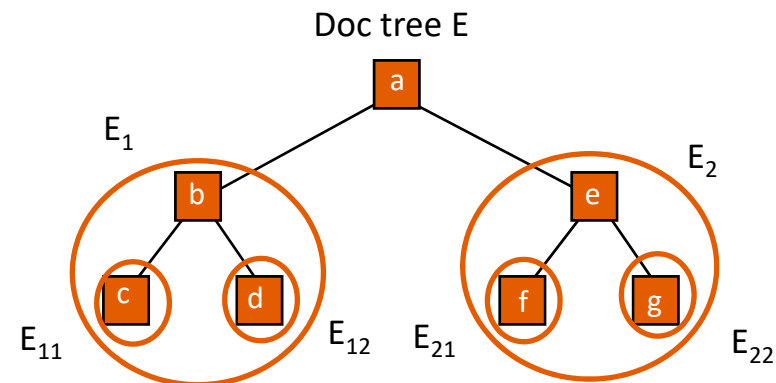
## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.1. Preliminaries

- **Sub-tree:** Given two trees  $T$  and  $T'$ ,  $T'$  is a sub-tree of  $T$  if all nodes of  $T'$  occur in  $T$ , with the same parent/child edge relationship and node order, such that no additional nodes occur in the embedding of  $T'$ 
  - The sub-trees of  $E$  are:  $E_1$ ,  $E_{11}$ ,  $E_{12}$ ,  $E_2$ ,  $E_{21}$ , and  $E_{22}$
- **First level sub-tree:** Given a tree  $T$  with root  $r$  of degree  $k$ , the first level sub-trees  $\text{FLS}(T) = \langle T_1, \dots, T_k \rangle$  of  $T$  are the sub-trees rooted at the children nodes of  $r$ :  $r_1, \dots, r_k$ 
  - $\text{FLS}(E) = \langle E_1, E_2 \rangle$



## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.2. Tree Edit Operations

- It uses three atomic node operations
  - An atomic operation is an edit operation applied on a single tree node
    - Leaf node insertion, noted: **Ins(n)** where **n** is a leaf node
    - Leaf node deletion, noted: **Del(n)**, where **n** is a leaf node
    - Node update: noted **Upd(n<sub>1</sub>, n<sub>2</sub>)** where **n<sub>1</sub>** and **n<sub>2</sub>** are inner/leaf nodes



Similar to *Chawathe's* operations

- As well as two complex tree operations
  - A tree operation is applied on a tree (or sub-tree)
    - Tree insertion, noted: **InsTree(SbT)** where **SbT** is a tree/sub-tree
    - Tree deletion, noted: **DelTree(SbT)** where **SbT** is a tree/sub-tree

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.2. Tree Edit Operations

- **Formal** edit operations **syntax:**
  - Refer to *Chawathe's* approach for formal **leaf node insertion**, **leaf node deletion** and **node update** operation syntaxes
  - **Tree Insertion:** Given a tree  $A$  and a tree  $T$  with an inner node  $p$  having first level sub-trees  $P_1, P_2, \dots, P_m$ ,  $\text{InsTree}(A, p, i)$  is a tree insertion applied to  $T$ , inserting  $A$  as the  $i^{\text{th}}$  sub-tree of  $p$ , thus yielding  $T'$  with first level sub-trees  $T_1, \dots, T_{i-1}, A, T_{i+1}, \dots, T_{m+1}$ .
  - **Tree deletion:** Given a tree  $A$  and a tree  $T$  with an inner node  $p$ ,  $A$  being the  $i^{\text{th}}$  sub-tree of  $p$ ,  $\text{DelTree}(A)$  is a tree deletion operation applied to  $T$  that yields  $T'$  with first level sub-trees  $T_1, \dots, T_{i-1}, T_{i+1}, \dots, T_m$

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.3. Tree Edit Operations Costs

- Different edit operation costs schemes can be used
  - **Intuitive cost model** for **atomic** edit **operations**:
    - $\text{Cost}_{\text{Ins}(n)} = \text{Cost}_{\text{Del}(n)} = 1$ 
      - Unit cost
    - $\text{Cost}_{\text{Upd}(n_1, n_2)} = \begin{cases} 1 & \text{if } n_1 \neq n_2 \\ 0 & \text{otherwise} \end{cases}$
  - **NOTE:** Different edit operation cost models can be chosen by the system designer
    - Based on the application domain, given theoretical/experimental foundations
      - For instance: we would consider the syntactic and/or semantic similarity between node labels and/or values, or node depth, etc.

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.3. Tree Edit Operations Costs

- Given two trees **A** (source tree) and **B** (destination tree) being compared

– **Intuitive cost model** for **complex** tree **operations**:

- $$\text{Cost}_{\text{InsTree}}(\text{SbT}) = \begin{cases} 1 & \text{if SbT is \textit{contained-in} the source tree A} \\ \sum_{n \in \text{SbT}} \text{Cost}_{\text{Ins}}(n) & \text{otherwise} \end{cases}$$

- $$\text{Cost}_{\text{DelTree}}(\text{SbT}) = \begin{cases} 1 & \text{if SbT is \textit{contained-in} the destination tree B} \\ \sum_{n \in \text{SbT}} \text{Cost}_{\text{Del}}(n) & \text{otherwise} \end{cases}$$

## Ch. 5 - Structural Similarity

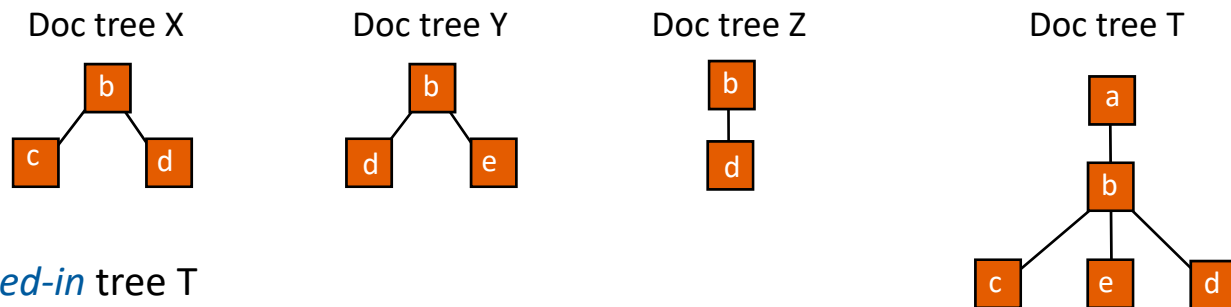
### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.4. Tree Contained-In Relation

- A tree pattern  $T_1$  is said to be **contained-in** a tree  $T_2$  if all nodes of  $T_1$  occur in  $T_2$ , with the same parent/child edge relationship and relative node order
  - Additional nodes can appear in  $T_2$

- Example:



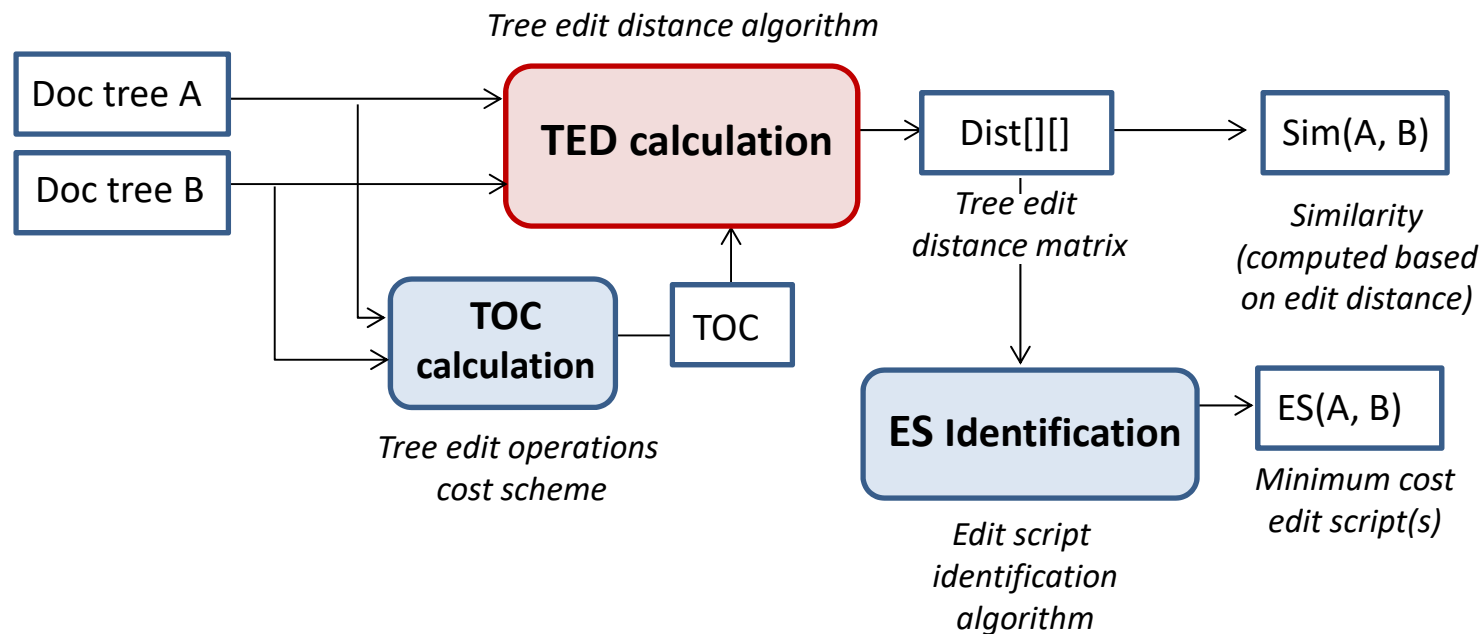
- Tree X is **contained-in** tree T
- Tree Y is **not contained-in** tree T
  - Since nodes of labels “d” and “e” have different *relative* node order in Y and T
- Tree Z is **contained-in** tree T
  - Additional nodes may appear in T, even as children of nodes mapped to Z

## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.5. Overall Activity Diagram



## Ch. 5 - Structural Similarity

```

Input: A and B      // XML document trees to be compared
Output: TED(A, B)  // Edit distance between A and B

Begin

    M = Degree(A)      // The number of first level sub-trees in A      1
    N = Degree(B)      // The number of first level sub-trees in B      2

    Dist [][] = new [0...M][0...N]                                     3
    Dist[0][0] = CostUpd(R(A), R(B))                                   4

    For (i = 1 ; i ≤ M ; i++) { Dist[i][0] = Dist[i-1][0] + CostDelTree(Ai) } 5
    For (j = 1 ; j ≤ N ; j++) { Dist[0][j] = Dist[0][j-1] + CostInsTree(Bj) } 6

    For (i = 1 ; i ≤ M ; i++)                                          7
    {                                                                    8
        For (j = 1 ; j ≤ N ; j++)                                      9
        {                                                              10
            Dist[i][j] = min{                                         11
                Dist[i-1][j-1] + TED(Ai, Bj)                        12
                Dist[i-1][j] + CostDelTree(Ai),                      13
                Dist[i][j-1] + CostInsTree(Bj)                       14
            }                                                         15
        }                                                            16
    }                                                                    17

    Return Dist[M][N]         // ≡ TED(A, B)                          18

End

```

*Nierman & Jagadish's main tree edit distance algorithm, 2002*

## Ch. 5 - Structural Similarity

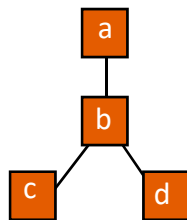
### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

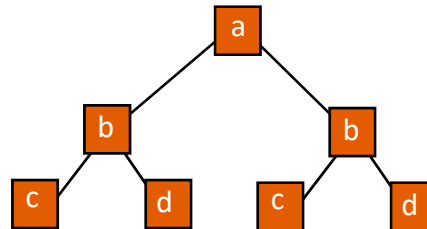
##### 3.3.6. Computing Tree Edit Distance

Example: Consider document trees C, D, and E

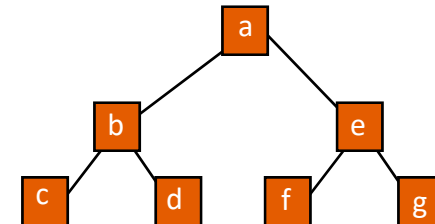
Doc tree C



Doc tree D



Doc tree E



- The tree will be processed in their native form
  - As nested sets of first level sub-trees
    - ⇒ **Multiple** nested **edit distance matrixes** will be **recursively** produced and processed

## Ch. 5 - Structural Similarity

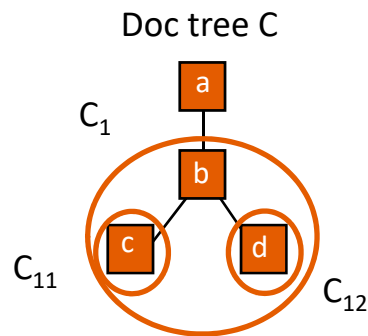
### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

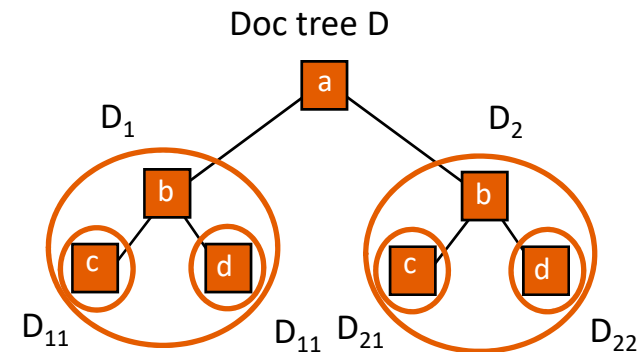
##### 3.3.6. Computing Tree Edit Distance

Example: Comparing document trees C (source tree) and D (destination tree)

- **Step 1:** Computing tree edit operations costs



$\text{Cost}_{\text{DelTree}}(C_1) = 1$  since  $C_1$  contained-in D  
 $\text{Cost}_{\text{DelTree}}(C_{11}) = \text{Cost}_{\text{DelTree}}(C_{12}) = 1$



$\text{Cost}_{\text{InsTree}}(D_1) = \text{Cost}_{\text{InsTree}}(D_2) = 1$  since  $D_1$  and  $D_2$  are contained-in C  
 $\text{Cost}_{\text{InsTree}}(D_{11}) = \text{Cost}_{\text{InsTree}}(C_{12}) = \text{Cost}_{\text{InsTree}}(C_{22}) = \text{Cost}_{\text{InsTree}}(C_{22}) = 1$

## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.6. Computing Tree Edit Distance

Example: Comparing document trees C (source tree) and D (destination tree)

- **Step 2:** Producing the first (main) edit distance matrix
  - Having **degree(C) = 1** and **degree(D) = 2**, we draw the following **edit distance matrix**
  - **Rows** designate the costs of **deleting** first level sub-trees from the source tree C
  - **Columns** designate the costs of **inserting** first level sub-trees into the destination tree D
  - **Diagonals** represent the costs of **updating** the labels of the root nodes of sub-trees being compared

**D →**

|            |                      |             |                      |                      |
|------------|----------------------|-------------|----------------------|----------------------|
|            |                      | <b>R(D)</b> | <b>D<sub>1</sub></b> | <b>D<sub>2</sub></b> |
| <b>C ↓</b> | <b>R(C)</b>          |             |                      |                      |
|            | <b>C<sub>1</sub></b> |             |                      |                      |

## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

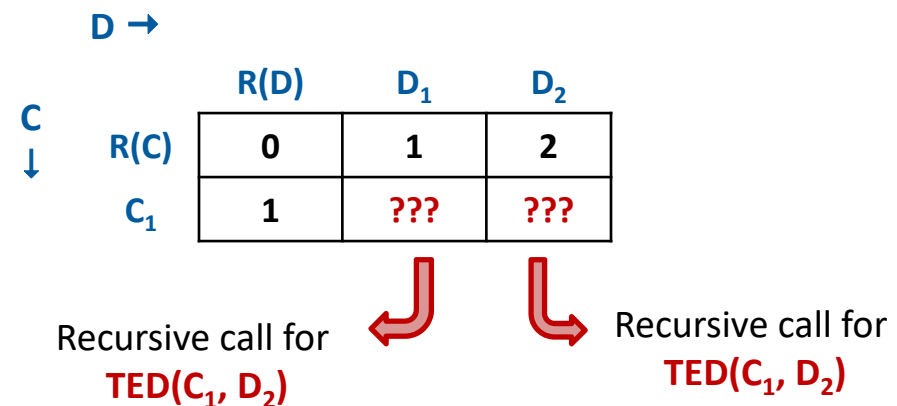
##### 3.3.6. Computing Tree Edit Distance

Example: Comparing document trees C (source tree) and D (destination tree)

- **Step 3:** Populating the first (main) edit distance matrix

- Running the edit distance algorithm populates the edit distance matrix as follows:

- A recursive call for the algorithm is launched when processing sub-trees  $C_1$  and  $D_2$



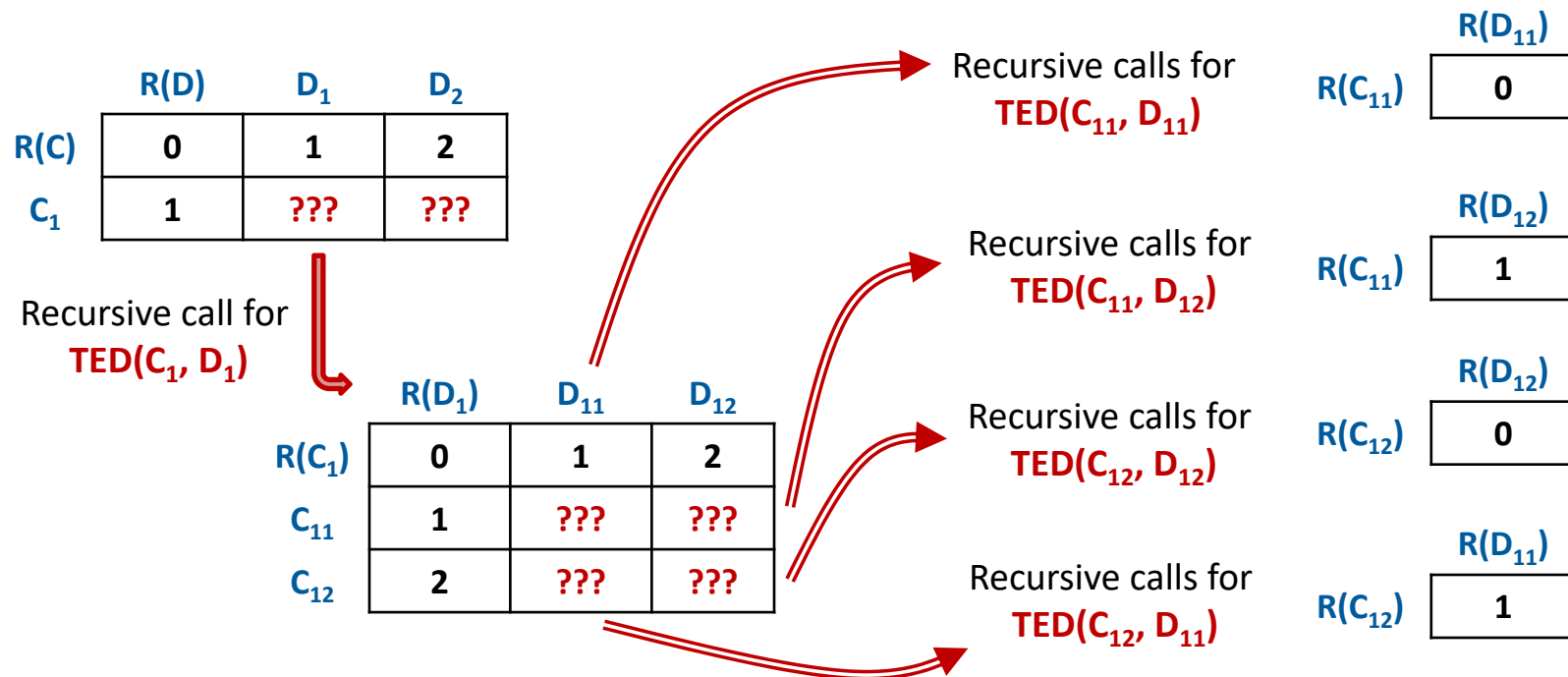
## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

Example: Comparing document trees C (source tree) and D (destination tree)

- Step 4:** Generating and populating recursive edit distance matrixes



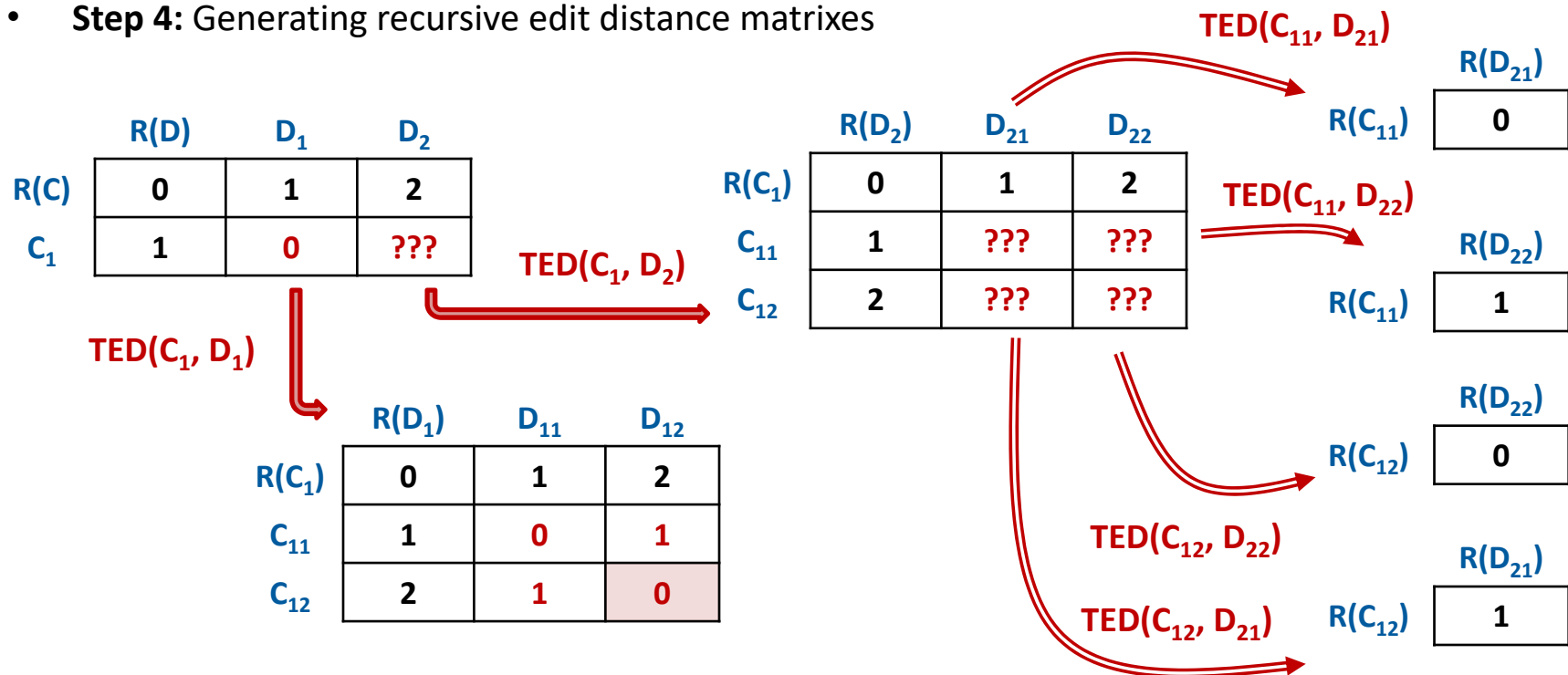
## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

Example: Comparing document trees C (source tree) and D (destination tree)

- Step 4:** Generating recursive edit distance matrixes



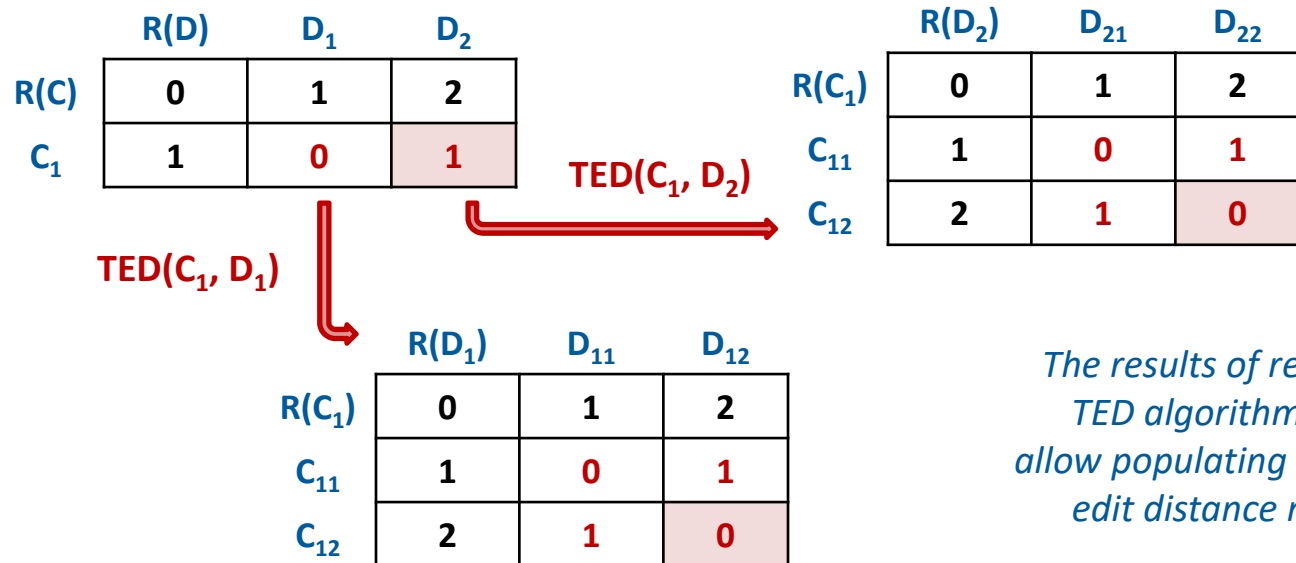
## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

Example: Comparing document trees C (source tree) and D (destination tree)

- **Step 4:** Generating and populating recursive edit distance matrixes



*The results of recursive TED algorithm calls allow populating the main edit distance matrix*

## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.6. Computing Tree Edit Distance

Example: Comparing document trees C (source tree) and D (destination tree)

– Given the main edit distance matrix in the previous slide

- The **minimum** edit script **cost** is: 1

$$\text{TED}(C, D) = \text{Min}(\text{Cost}_{\text{ES}(C, D)}) = \text{Dist}[\text{degree}(C), \text{degree}(D)] = \text{Dist}[1, 2] = 1$$

- The cost of edit operations **necessary** to transform tree C to D



How to identify these operations?



## Ch. 5 - Structural Similarity

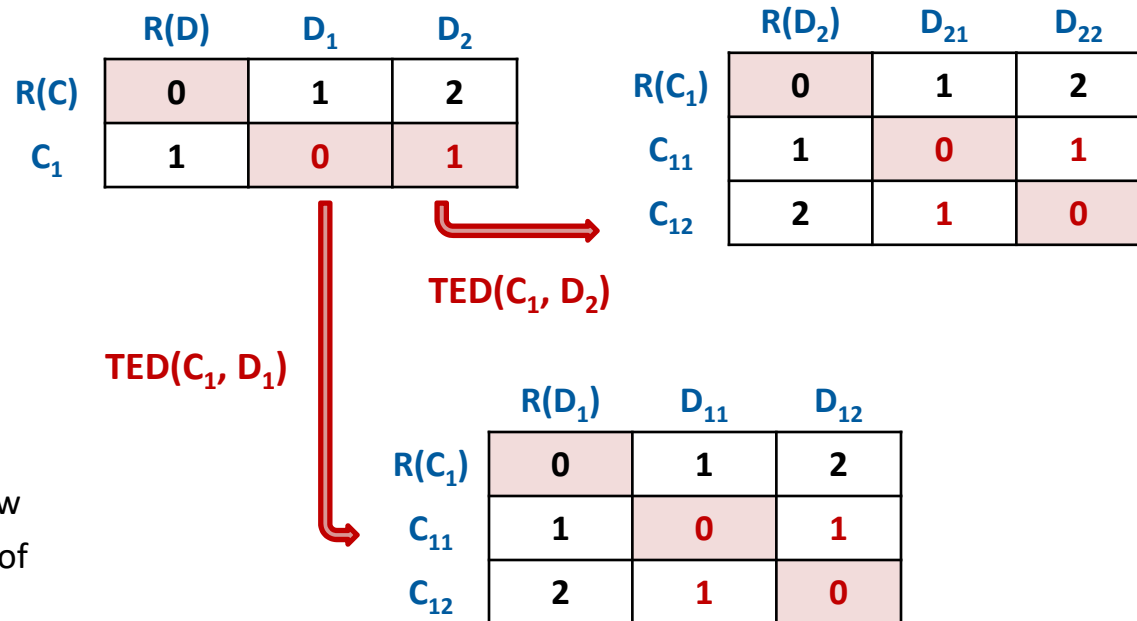
### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.7. Edit Script Identification

Example: Comparing document trees C (source tree) and D (destination tree)

- We post-process the populated edit distance matrixes, going from the main matrix to the recursive ones, to identify the **minimum cost edit script(s)** shown here as highlighted cells
- In this figure, we do not show single cell matrixes for ease of presentation



## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.7. Edit Script Identification

Example: Comparing document trees C (source tree) and D (destination tree)

– In this example, we identify two minimum cost edit scripts:

- **Scrip #1:**

- **Raw edit script:** navigating from the main ED matrix to recursive ones

$$ES_1(C, D) = \langle \text{InsTree}(D_2), \text{TED}(C_1, D_1) \rangle$$

$$\rightarrow ES(C_1, D_1) = \langle \text{TED}(C_{12}, D_{12}), \text{TED}(C_{11}, D_{11}), \text{Upd}(R(C_1), R(D_1)) \rangle$$

$$\rightarrow ES(C_{11}, D_{11}) = \langle \text{Upd}(R(C_{11}), R(D_{11})) \rangle$$

$$\rightarrow ES(C_{12}, D_{12}) = \langle \text{Upd}(R(C_{12}), R(D_{12})) \rangle$$

$$ES_1(C, D) = \langle \text{InsTree}(D_2), \text{Upd}(R(C_{12}), R(D_{12})), \text{Upd}(R(C_{11}), R(D_{11})), \text{Upd}(R(C_1), R(D_1)) \rangle$$

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.7. Edit Script Identification

Example: Comparing document trees C (source tree) and D (destination tree)

– **Scrip #1:**

- **Raw edit script:** navigating from the main ED matrix to recursive ones

$$ES_1(C, D) = \langle \text{InsTree}(D_2), \text{Upd}(R(C_{12}), R(D_{12})), \text{Upd}(R(C_{11}), R(D_{11})), \text{Upd}(R(C_1), R(D_1)) \rangle$$

- **Final edit script:** obtained after i) eliminating costless (useless) edit operations, and ii) inverting the order of edit operations

$$ES_1(C, D) = \langle \text{InsTree}(D_2) \rangle$$

⇒ Using the simplified syntax of the tree insertion operation

**Formally:**  $ES_1(C, D) = \langle \text{InsTree}(D_2, R(C), 2) \rangle$  where  $R(C)$  represents root node  $C[0]$  in tree C

⇒ Including parent node and sibling position at which the sub-tree should be inserted

## Ch. 5 - Structural Similarity

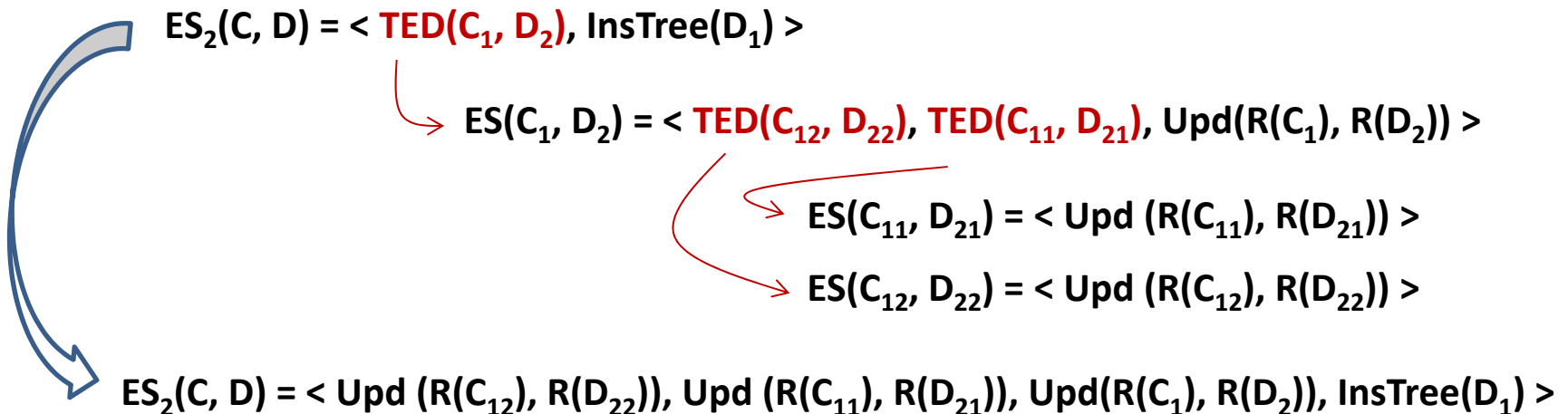
### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.7. Edit Script Identification

Example: Comparing document trees C (source tree) and D (destination tree)

- **Scrip #2:** we follow the same process used in identifying the first script
  - **Raw edit script:** navigating from the main ED matrix to recursive ones



## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.7. Edit Script Identification

Example: Comparing document trees C (source tree) and D (destination tree)

– **Script #2:** we follow the same process used to identifying the first script

- **Raw edit script:** navigating from the main ED matrix to recursive ones

$$ES_2(C, D) = < \text{Upd}(R(C_{12}), R(D_{22})), \text{Upd}(R(C_{11}), R(D_{21})), \text{Upd}(R(C_1), R(D_2)), \text{InsTree}(D_1) >$$

- **Final edit script:** obtained after i) eliminating costless (useless) edit operations, and ii) inverting the order of edit operations

$$ES_2(C, D) = < \text{InsTree}(D_1) >$$

⇒ Using the simplified syntax of the tree insertion operation

**Formally:**  $ES_2(C, D) = < \text{InsTree}(D_1, R(C), 1) >$  where  $R(C)$  represents root node  $C[0]$  in tree C

⇒ Including parent node and sibling position at which the sub-tree should be inserted

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.8. Similarity Computation

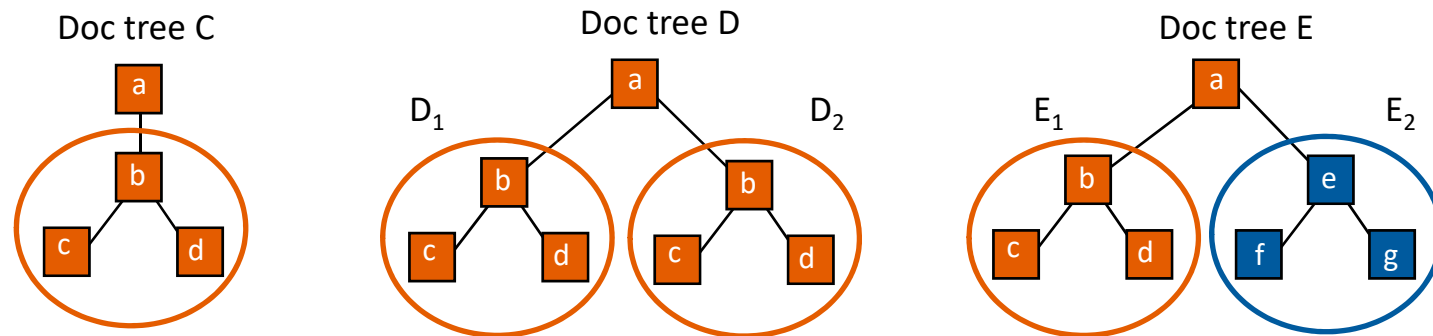
- We note *N&J*'s similarity as  $\text{Sim}_{N\&J}(\mathbf{A}, \mathbf{B})$  to distinguish it from  $\text{Sim}_{\text{Chawathe}}(\mathbf{A}, \mathbf{B})$ 
  - Which is computed based on  $\text{TED}_{N\&J}(\mathbf{A}, \mathbf{B})$ 
    - Where A and B are two document trees being compared
- Given input document trees C and D considered in our running example:
  - Formula 1:  $\text{Sim}_{N\&J}(\mathbf{C}, \mathbf{D}) = \frac{1}{1 + \text{TED}_{N\&J}(\mathbf{C}, \mathbf{D})} = \frac{1}{1 + 1} = 0.5$
  - Formula 2:  $\text{Sim}_{N\&J}(\mathbf{C}, \mathbf{D}) = 1 - \frac{\text{TED}_{N\&J}(\mathbf{C}, \mathbf{D})}{|\mathbf{C}| + |\mathbf{D}|} = 1 - \frac{1}{4 + 7} \approx 0.9091$
- **NOTE**: Comparing similarity functions, to decide which one to use, is covered in Ch. 4

## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

Example: Considering document trees C, D, and E



- $\text{Sim}_{\text{N\&J}}(\mathbf{C}, \mathbf{D}) = 0.5$  having  $\text{TED}_{\text{N\&J}}(\mathbf{C}, \mathbf{D}) = 1$  and using similarity *Formula 1*
  - Cost of  $\text{ES}_1(\mathbf{C}, \mathbf{D}) = \langle \text{InsTree}(\mathbf{D}_2) \rangle$  or  $\text{ES}_2(\mathbf{C}, \mathbf{D}) = \langle \text{InsTree}(\mathbf{D}_1) \rangle$
- $\text{Sim}_{\text{N\&J}}(\mathbf{C}, \mathbf{E}) = 0.25$  having  $\text{TED}_{\text{N\&J}}(\mathbf{C}, \mathbf{E}) = 3$  and using similarity *Formula 1*
  - Cost of  $\text{ES}(\mathbf{C}, \mathbf{E}) = \langle \text{InsTree}(\mathbf{E}_2) \rangle$



$$\text{Sim}_{\text{N\&J}}(\mathbf{C}, \mathbf{D}) > \text{Sim}_{\text{N\&J}}(\mathbf{C}, \mathbf{E})$$

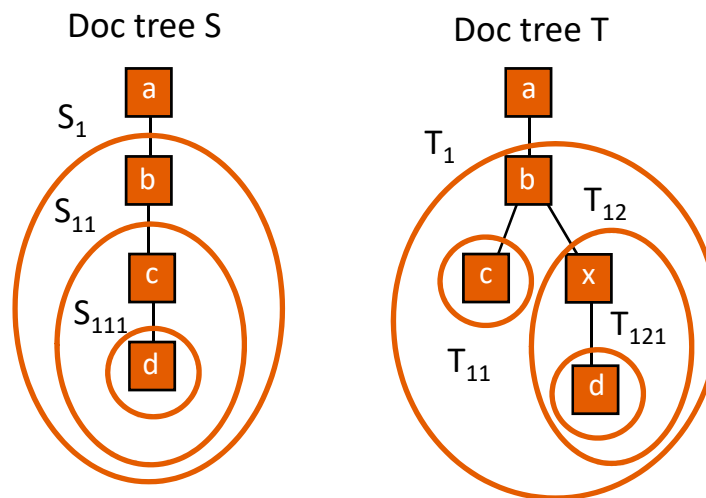
*Sub-tree similarities effectively detected...*

## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

- Consider for instance document document trees S and T:



$$\text{TED}_{\text{N\&J}}(S, T) = 2$$

$$\text{ES}_1(S, T) = \langle \text{InsTree}(T_{11}), \text{Upd}(R(S_{11})) \rangle$$

$$\text{or } \text{ES}_2(S, T) = \langle \text{Upd}(R(S_{11})), \text{InsTree}(T_{11}) \rangle$$

Producing accurate results due to processing the trees as trees considering their structural properties

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.9. Complexity Analysis

- The **time complexity** of *N&J*'s algorithm is :  
 $O(\text{degree}(A) + \text{degree}(B) + \text{degree}(A) \times \text{degree}(B) \times O(TED_{Rec}))$ 
  - Where  $O(TED_{Rec})$  represents the complexity of the algorithm's recursive calls
    - Which simplifies to  $O(|A| \times |B|)$  since the algorithm will be recursively called once for each pair of vertices at the same depth in the input trees *A* and *B*
      - ⇒ Overall algorithm complexity simplifies to  **$O(N^2)$** 
        - With  $N = \max\{|A|, |B|\}$
  - The **space complexity** of *N&J*'s algorithm is also  $O(|A| \times |B|)$ 
    - Which corresponds to the memory size required to store all edit distance matrixes
  - The algorithm is of **quadratic** complexity with respect to tree size
    - It is linearly dependent on the size of each tree

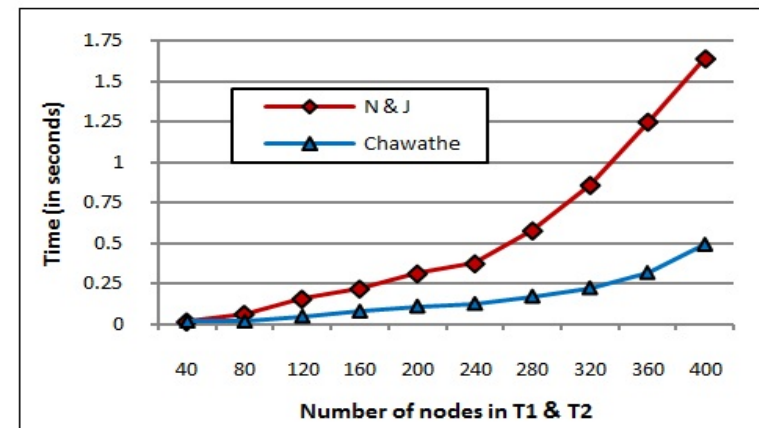
## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.9. Complexity Analysis

- Note that while both *N&J* and *Chawathe*'s approaches have the same complexity levels
  - Nonetheless, *N&J*'s process induces additional **overhead** due to:
    - *N&J*'s recursive calls for every pair of nodes at the same level in the trees
    - Computing tree edit operations costs and evaluating the *contained-in* relation for every sub-tree in the source and destination trees being compared



*Time experiments reported from [1]*

[1] Tekli et al., A Novel XML Structure Comparison Framework based on Sub-tree Commonalities and Label Semantics, Elsevier Journal of Web Semantics (JWS): Science, Services, and Agents on the World Wide Web, 11: 14-40, 2012.

## Ch. 5 - Structural Similarity

---

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.10. Discussion

- **N&J** allows **detecting structural similarities** between document trees
  - Which were previously missed by *Chawathe's* approach
- **N&J** allows **preserving structural properties** of document trees during comparison
  - Which were partially disregarded by *Chawathe's* approach
- **N&J** shares the **same complexity levels** with *Chawathe's* approach
  - Yet induces a significant time **overhead**
- Certain tree structural similarities remain undetected using **N&J's** method
  - Repetitions of structurally similar sub-trees
  - Repetitions of leaf node sub-trees, among other cases...

⇒ As briefly described in the following examples

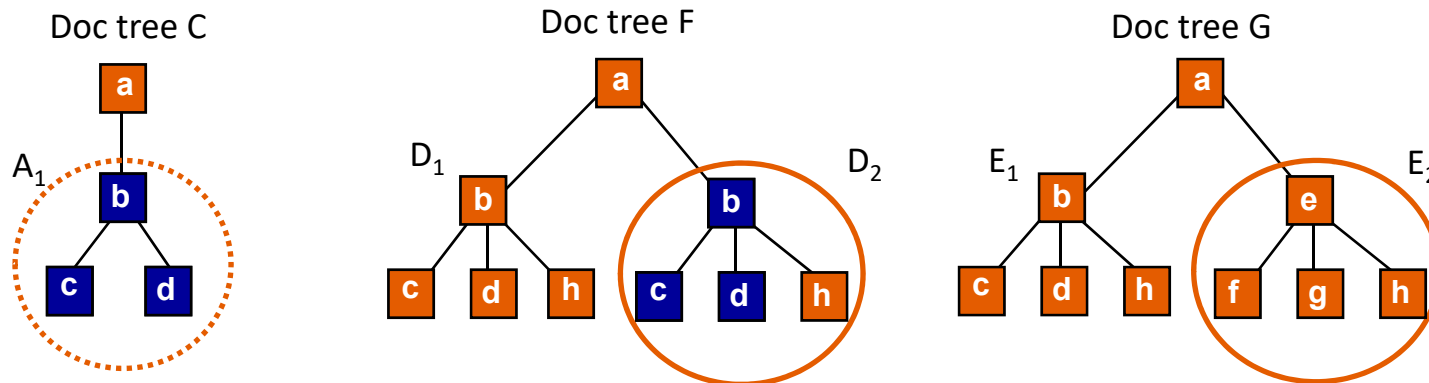
## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.10. Discussion

Case 1: Considering document trees C, D, and E



$$\text{TED}_{\text{N\&J}}(C, F) = \text{TED}_{\text{N\&J}}(C, G) = 5$$



The ***contained-in*** relations are not fulfilled!

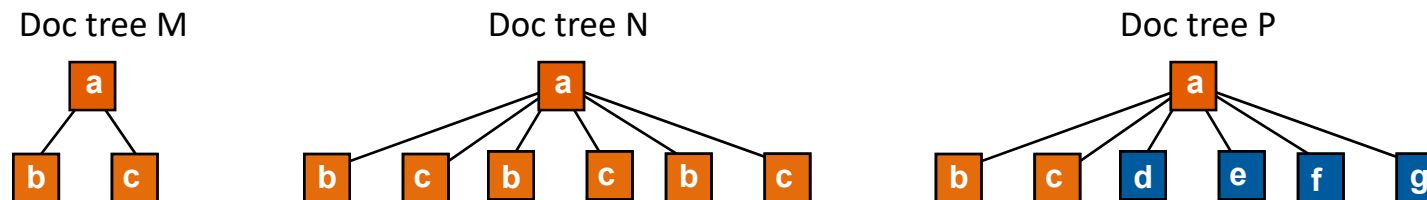
## Ch. 5 - Structural Similarity

### 3. Tree Edit Distance

#### 3.3. Nierman & Jagadish Approach

##### 3.3.10. Discussion

Case 2: Considering document trees M, N, and P



$$\text{TED}_{\text{N\&J}}(M, N) = \text{TED}_{\text{N\&J}}(M, P) = 4$$



Unable to detect leaf node repetitions...

⇒ Undetected similarities are addressed in subsequent solutions, namely [1]

[1] Tekli et al., A Novel XML Structure Comparison Framework based on Sub-tree Commonalities and Label Semantics, Elsevier Journal of Web Semantics (JWS): Science, Services, and Agents on the World Wide Web, 11: 14-40, 2012.

## Ch. 5 - Structural Similarity

---

### 4. Applications of TED-based Similarity

- The use of semi-structured similarity ranges over a wide spectrum of applications :
  - Semi-structured **data versioning** and **data warehousing**
    - Querying the past, querying and monitoring changes, archiving and mirroring
  - Semi-structured **data classification**
  - Semi-structured **data clustering**
  - Semi-structured **data integration**
  - Approximate semi-structured **data querying**
  - **Selective dissemination** of semi-structured data
  - **Alert filtering** in intrusion detection systems
  - **Web Service matching** and SOAP/REST processing
  - ...

## Ch. 5 - Structural Similarity

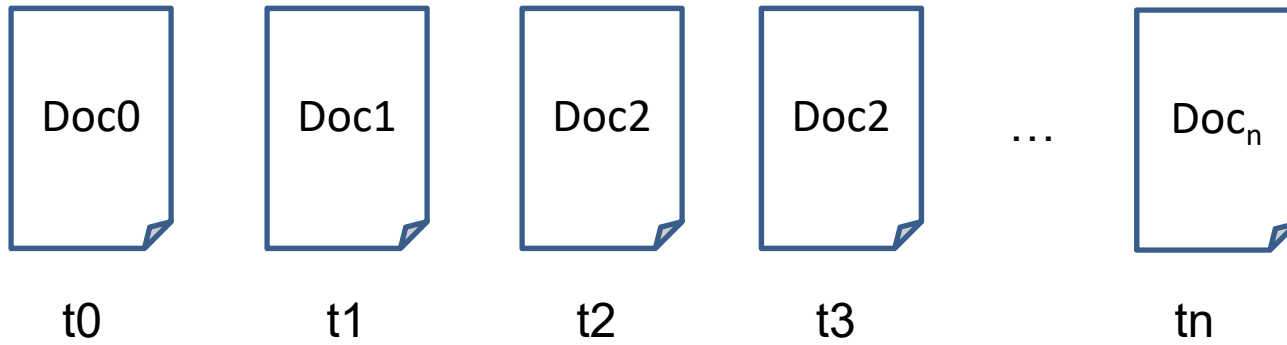
---

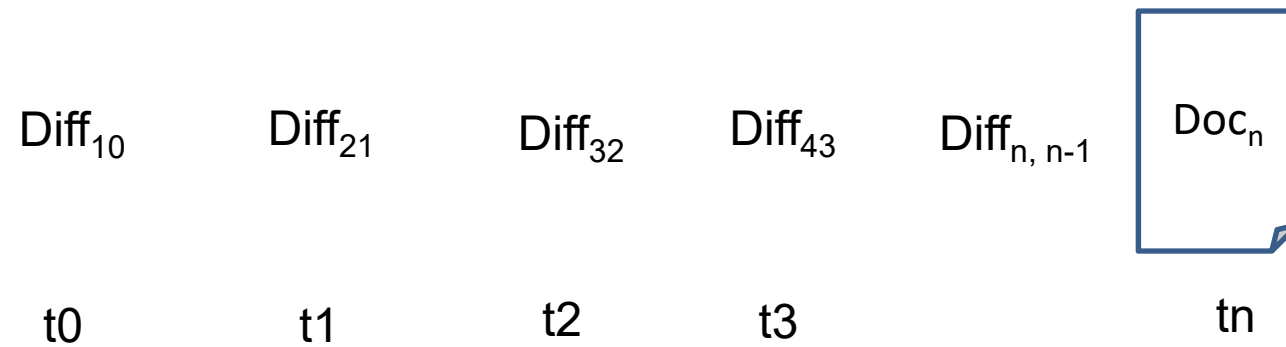
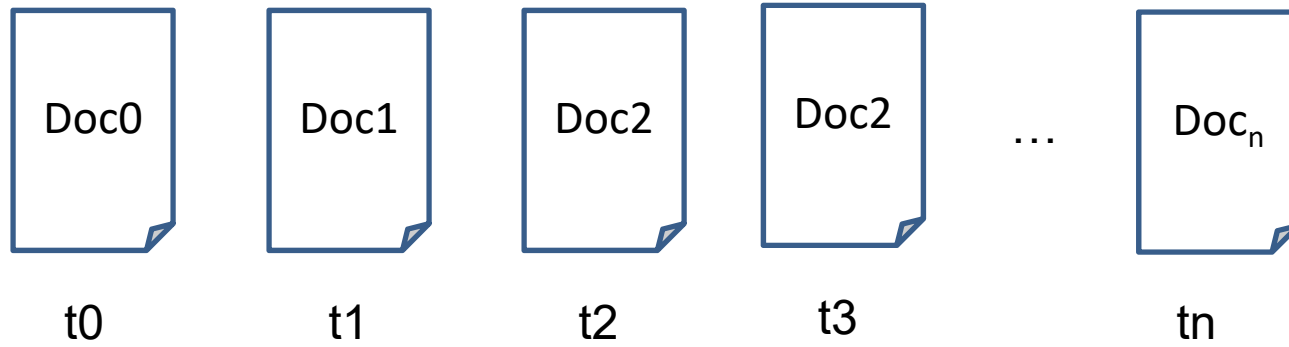
### 4. Applications of TED-based Similarity

#### 4.1. Data Versioning and Data Warehousing

- **Data warehouse:** central repository of integrated data from one or more disparate data sources (e.g., databases)
  - It stores current and historical data for trend reporting and data analysis
- **XML versioning:** storing different versions of the semi-structured data [1]
  - Support for the control of changes in a warehouse of semi-structured documents
- Main requirement: a TED-based measure
  - Which is also identified here as a *differencing algorithm*, or *diff*
    - A similarity measure that provides edit scripts, also identified here as *deltas* or *diffs*
- **Deltas/Diffs** offer means for **change detection** and **representation** between documents
  - Resembling traditional **logs** in **database** systems

[1] Marian A.; Abiteboul S. and Mignet L., *Change-Centric Management of Versions in an XML Warehouse. Proceedings of the International Conference on Very Large Data Bases (VLDB)*, pp. 581-590, 2001.





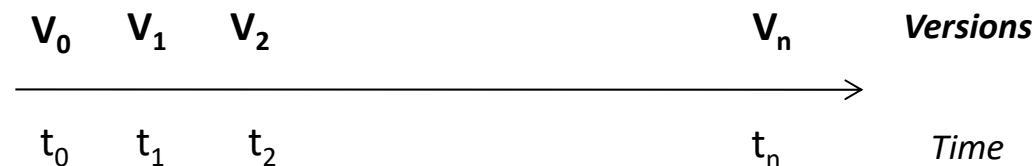
## Ch. 5 - Structural Similarity

---

### 4. Applications of TED-based Similarity

#### 4.1. Data Versioning and Data Warehousing

- **Data versioning:** storing different versions of the data generated at different times
  1. Basic approach: storing different versions of a semi-structured document (grammar)
    - Given document **D** created at time  $t_0$ , which we designate as version  $V_0$



- **Pro:** Easy access to versions produced at any time interval
- **Cons:**
  - Need for huge storage capacity
  - Data redundancy, which can slow down data access

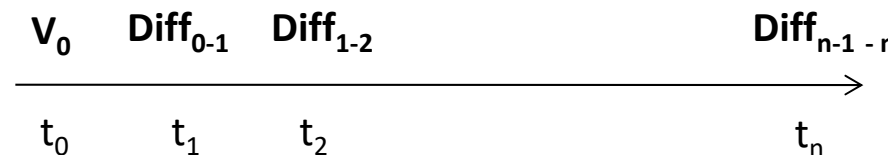
## Ch. 5 - Structural Similarity

### 4. Applications of TED-based Similarity

#### 4.1. Data Versioning and Data Warehousing

- There exist different strategies to perform differencing:
  2. Storing the original (oldest) version of the document, with subsequent deltas

- The delta between versions  $V_i$  and  $V_j$ ,  $\text{Diff}(V_i, V_j)$ , is noted  $\text{Diff}_{i-j}$



- **Patching** is applied to produce version  $V_i$  based on original version  $V_0$ 
    - By **composing** corresponding deltas, i.e.,  $\text{Diff}_{0-i} = \text{Diff}_{0-1} \oplus \text{Diff}_{1-2} \oplus \dots \oplus \text{Diff}_{i-1-i}$
  - **Pro:** Intuitive straightforward solution based on TED logic
  - **Cons:**
    - Additional processing required to patch original version with deltas
    - Additional processing required to compose deltas
- ⇒ Impeding both data **production** and **access** time

## Ch. 5 - Structural Similarity

### 4. Applications of TED-based Similarity

#### 4.1. Data Versioning and Data Warehousing

- Differencing approach: using a **TED**-based solution, following different strategies:

#### 3. Storing the latest (newest) version of the document, with previous deltas

- The delta between versions  $V_i$  and  $V_j$ ,  $\text{Diff}(V_i, V_j)$ , is noted  $\text{Diff}_{i-j}$



- **Pros:**
  - Direct access to latest version of the data (needed most frequently)
  - Less time to compute deltas (no need for unnecessary patching)
- **Con:** Accessing earlier versions requires computing the inverse of the delta, which:
  - Requires additional processing time
  - Is not straightforward since it involves a risk of information loss
 ⇒ **Bi-directional deltas** are required

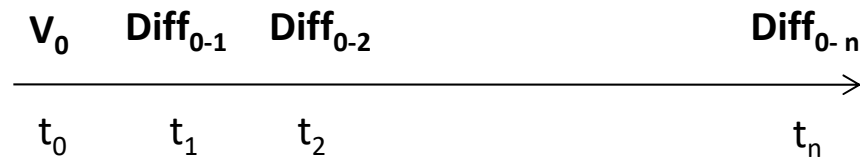
## Ch. 5 - Structural Similarity

### 4. Applications of TED-based Similarity

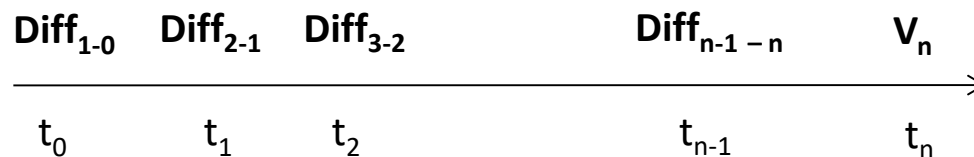
#### 4.1. Data Versioning and Data Warehousing

- Differencing approach: using a **TED**-based solution, following different strategies:
  - Other strategies can be considered, e.g.:

#### 4. Storing the **initial version** and **composite deltas**



#### 5. Storing the **latest version** with **backward deltas**



## Ch. 5 - Structural Similarity

---

### 4. Applications of TED-based Similarity

#### 4.1. Data Versioning and Data Warehousing

##### 4.1.1 Temporal Querying

- One may want to view or access a particular version of a document
  - Which was available at a particular time
    - For instance, an earlier version of a web page
- Standard use of versions, namely **recording history**
  - Obtain an old version of a document, by asking a **query** about **the past**
    - ⇒ Ask for the value of some semi-structured data element at some previous time
- The **deltas** can be **stored as semi-structured documents**
  - ⇒ Temporal queries become regular queries over documents
    - Different semi-structured diff output formats exist: *DeltaXML*, *XyDiff*, and *SDL* [1]

[1] Tekli J., Damiani E., and Chbeir R., *Using XML-Based Multicasting to Improve Web Service Scalability*. International Journal on Web Service Research, 9(1): 1-29 (2012)

## Ch. 5 - Structural Similarity

---

### 4. Applications of TED-based Similarity

#### 4.1. Data Versioning and Data Warehousing

##### 4.1.2 Querying and Monitoring Changes

- One may want to **query changes** applied to the document
  - E.g., ask for the list of items recently introduced in a document
    - ⇒ **Querying the deltas** themselves
- **Detecting changes** of interest in semi-structured documents
  - E.g., that an element has been added to a document, or has been deleted
  - At the time the delta is computed, the system verifies for each atomic change whether this change is monitored by some subscription
    - E.g., the insertion of a new item in the document, a deletion of a given item, etc.
  - ⇒ Basis for **query subscription** and **notification systems** [1]

[1] Nguyen B.; Abiteboul S.; Cobena G. and Preda M., *Monitoring XML Data on the Web*. Proceedings of the ACM International Conference on Management of Data (SIGMOD), 2001. pp. 437-448.

## Ch. 5 - Structural Similarity

---

### 4. Applications of TED-based Similarity

#### 4.1. Data Versioning and Data Warehousing

##### 4.1.3 Mirroring and Archiving

- Popular Web and FTP servers often have dozens of **mirror sites** around the world
  - **Changes** made to the master server **need to be propagated to** the **mirror** sites
  - Basic approach: users or programs making changes keep a record of updated data
    - ⇒ Need for **diff** algorithms that **compute and propagate only the difference** between the data version at the server and that at a mirror site
      - Reduce the amount of data transmitted over a network
- **Archiving** using TED approaches means storing the sequence of deltas before a certain date to archive corresponding semi-structured data
  - ⇒ Saving storage space

## Ch. 5 - Structural Similarity

---

### 4. Applications of TED-based Similarity

#### 4.2. Classification

- **Classification**, a.k.a *categorization* or *labeling*: identifying an output value based on input data, by **mapping inputs to desired outputs** (categories or labels)
  - Determining what class (category or label) an entity belongs to
    - Usually after analyzing a number of (pre-classified) examples
- Scenario: consider a number of heterogeneous databases that exchange semi-structured documents with each other on the Web
  - Each database representing and indexing the local documents according to a set of **predefined grammars**
    - **Documents** exchanged/introduced in a given database **are matched**, via **structural similarity**
      - **Against the local grammars** in the database

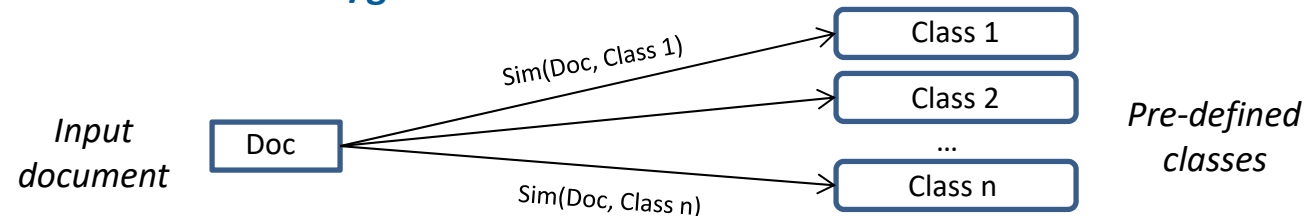
## Ch. 5 - Structural Similarity

### 4. Applications of TED-based Similarity

#### 4.2. Classification

##### Main process:

- Compare an input document with each **class**, which can be **described using**:
  - A representative **semi-structured document**
  - A representative document **grammar**
  - A **group of semi-structured docs/grammars**



- The match resulting in the highest similarity value is considered as: **best match**
  - The input document is then added to the corresponding class
- Considering a similarity threshold:  $Thresh_{Sim}$ 
  - If  $\forall Sim(Doc, Class_i) < Thresh_{Sim}$ , then a new class is created, made of the input document

## Ch. 5 - Structural Similarity

---

### 4. Applications of TED-based Similarity

#### 4.2. Classification

- Different **classification algorithms** exist in the literature [1]
  - A.k.a. **supervised learning** algorithms
    - **Instance-based learning** (involving similarity computation)
      - E.g., *k nearest neighbor* (kNN) algorithm
    - **Decision Lists and Trees**
    - **Support Vector Machines**
    - **Bayesian networks**
    - **Neural networks**
    - **Ensemble methods**
      - Combining multiple (weak) classifiers to form a more sophisticated one
        - E.g., *majority voting, probability mixture, rank-based combination*

Covered in more detail in  
***Intelligent Engineering Algorithms***  
course (COE 544/744)

## Ch. 5 - Structural Similarity

---

### 4. Applications of TED-based Similarity

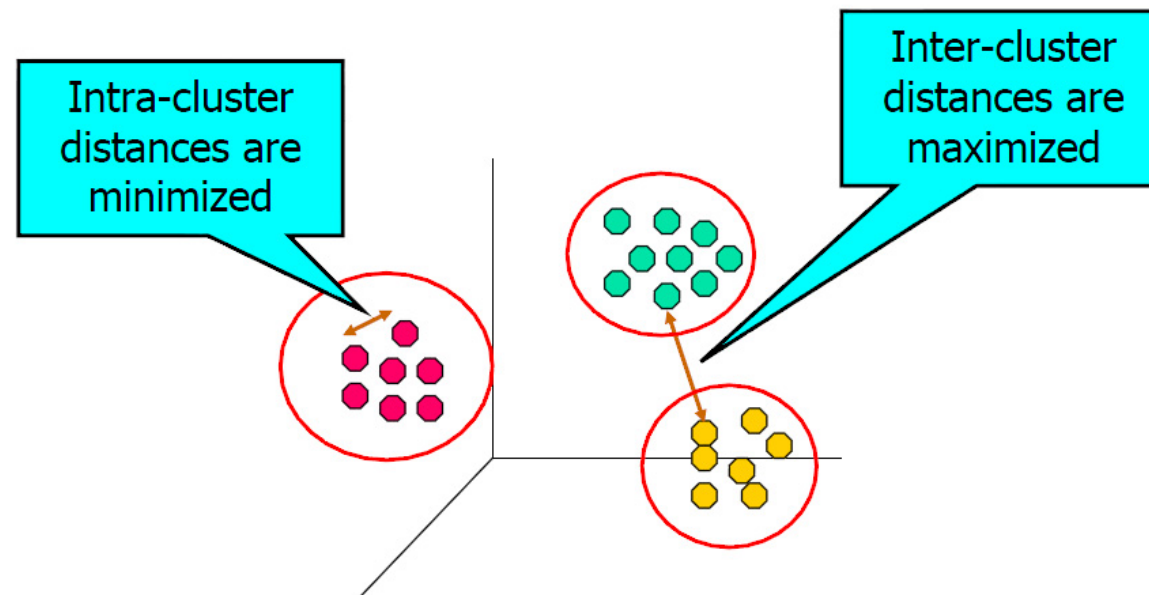
#### 4.3. Clustering

- **Clustering**, a.k.a. unsupervised learning: discovering and describing the organization of input data with no a priori information on the output
  - It can be viewed as classification with no pre-defined categories
- Scenario: Organizing a repository of heterogeneous semi-structured documents (grammars) into sub-groups, i.e., clusters, having similar characteristics
  - Grouping similar documents together can **improve** data **storage indexing**
    - And thus **positively affect** the **retrieval process**
      - If two documents/elements are similar, it is likely that they both either satisfy or not a given query
      - Therefore, when grouped together, similar documents/elements would be much easier to retrieve than when scattered at different locations in the storage device

## Ch. 5 - Structural Similarity

### 4. Applications of TED-based Similarity

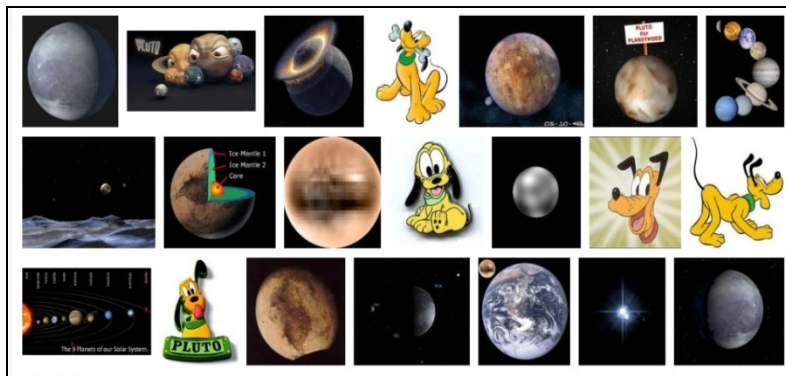
#### 4.3. Clustering



Why would we want to cluster?  
Any real applications?

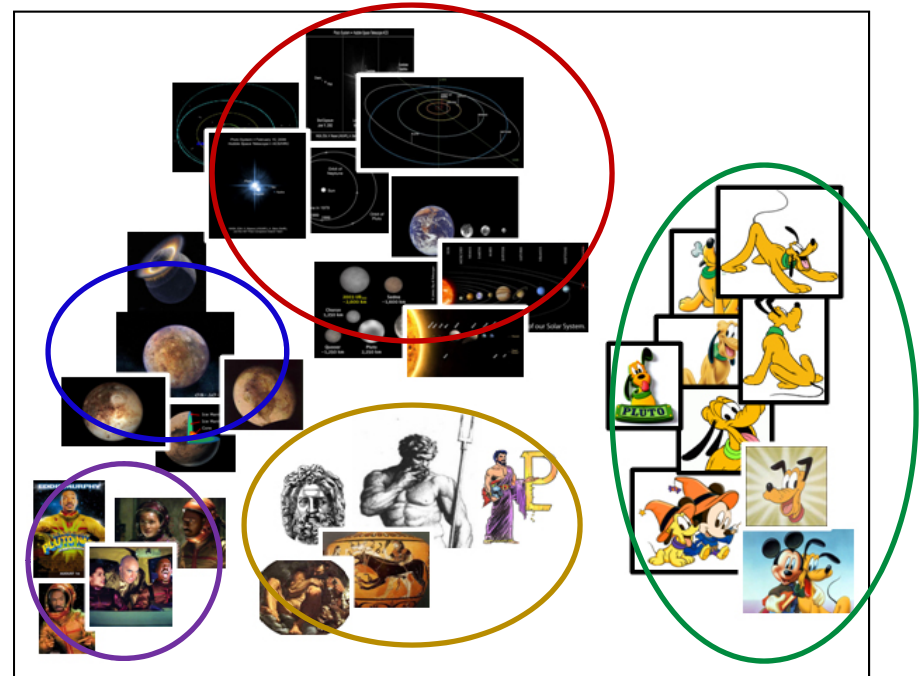
### 4.3. Clustering

- Example: Image clustering
  - Based on low-level features such as: **color**, **texture**, and **shape**



Input unlabeled data

*Without any  
previous knowledge  
about data organization*



Data clustered in 5 groups

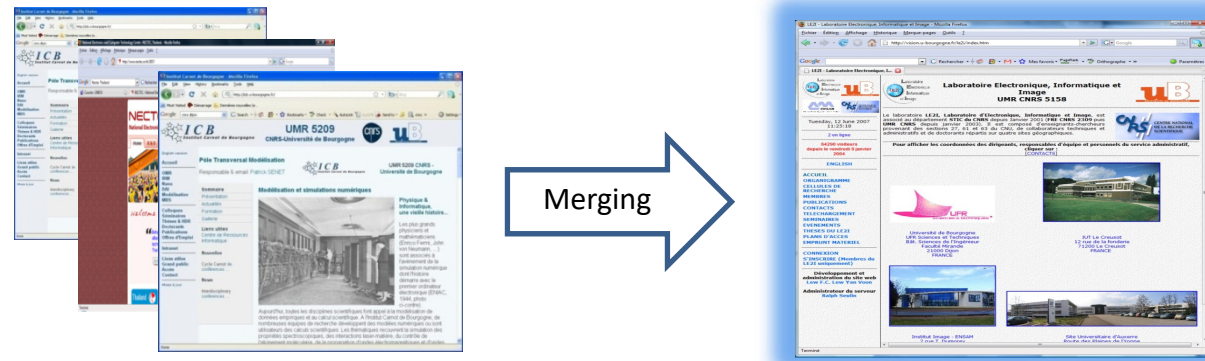
⇒ Identifying 5 clusters of similar images

## Ch. 5 - Structural Similarity

### 4. Applications of TED-based Similarity

#### 4.4. Data Integration

- Semi-structured documents from **different data sources** might contain **nearly or exactly the same information** but might be constructed using different structures
  - Even if two documents express similar contents, each of them may have some extra information w.r.t. to the other
- ⇒ Need for an effective similarity measure to **integrate** such **data sources**
  - So that the user can easily access and acquire more complete information



## Ch. 5 - Structural Similarity

---

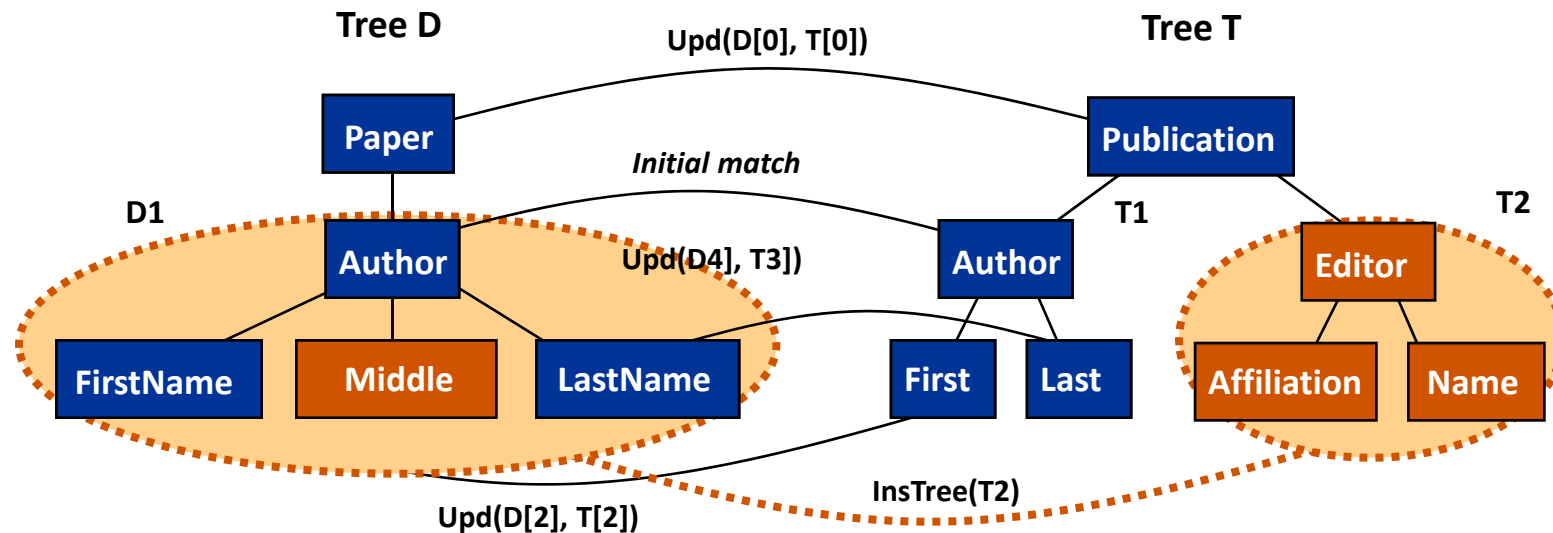
### 4. Applications of TED-based Similarity

#### 4.4. Data Integration

- Data integration task consists of two main steps:
  1. Similarity evaluation: performing an **approximate join** between data
    - Given two data sources,  $S_1$  and  $S_2$ , a similarity threshold  $\text{Thresh}_{\text{Sim}}$ , and a similarity function  $\text{Sim}(d_1, d_2)$  between two documents  $d_1$  (from  $S_1$ ) and  $d_2$  (from  $S_2$ )
    - Output mapping between all pairs of documents  $(d_1, d_2)$  such that  $\text{Sim}(d_1, d_2) \geq \text{Thresh}_{\text{Sim}}$
  2. Data merging: Paired documents are merged together to form unified views
    - Differencing (TED) techniques are particularly useful here
      - Processing the edit script (diff/delta) to **identify commonalities and differences** between the paired documents
      - These provide a **mapping between document trees** used to perform merging
        - Given predefined **merging rules** (e.g., given two identical nodes, eliminate one and preserve the other)

## Ch. 5 - Structural Similarity

Example: Producing mapping based on edit script (diff/delta)

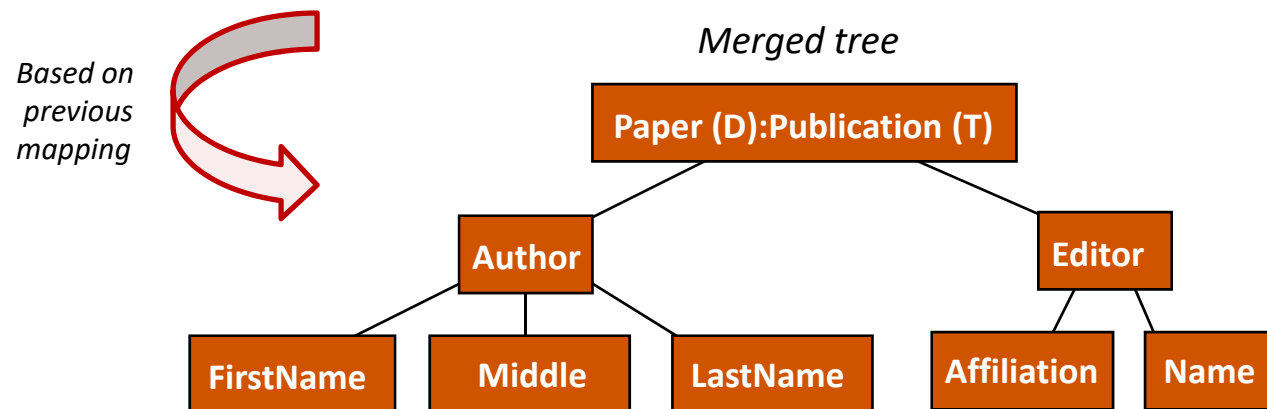


| Local match cardinality | Elements of tree D                                | Elements of tree T                          | Mapping Scores |
|-------------------------|---------------------------------------------------|---------------------------------------------|----------------|
| 1:1                     | D[0] (D[0].l= 'Paper')                            | T[0] (B[0].l='Publication')                 | 0.1667         |
|                         | D[1] (D[1].l= 'Author')                           | T[1] (T[1].l='Author')                      | 1              |
|                         | D[2] (D[2].l= 'FirstName')                        | T[2] (T[2].l='First')                       | 0.8519         |
|                         | D[4] (D[4].l= 'LastName')                         | T[3] (T[3].l='Last')                        | 0.8571         |
| n:n                     | D[1], D[2], D[3], D[4] (sub-tree D <sub>1</sub> ) | T[4], T[5], T[6] (sub-tree T <sub>2</sub> ) | 0.4142         |

## Ch. 5 - Structural Similarity

### Example: Sample merging rules

- **R1:** Given matching nodes with identical labels, preserve one and eliminate the other
- **R2:** Given matching nodes with different labels, and given that one label is syntactically included in another, preserve the encompassing label and eliminate the included one
- **R3:** Given matching nodes with different labels, and given that the semantic meaning of one label encompasses (is more general than) the semantic meaning of the other, preserve the more general label and eliminate the more specific one
- ...



## Ch. 5 - Structural Similarity

---

### 4. Applications of TED-based Similarity

#### 4.5. Approximate Querying

- Efficient retrieval of semi-structured documents becomes more and more important
  - With the increasing use of semi-structured data, specifically on the Web
- The DB community has proposed several languages for querying semi-structured data
  - Including **XML-QL**, **XQL** and **XQuery**
    - ⇒ Based on exact matching and do not support ranked queries
- Several attempts have been made to extend these query languages, **using similarity-based processing**, in order to **allow ranked results**
  1. Comparing query data (documents/fragments/elements) with document collection
  2. Returning data which similarity with query data is above user chosen  $Thresh_{sim}$
  3. Results are ranked based on similarity



Covered in more detail in *Chapter 5*

## Ch. 5 - Structural Similarity

---

**Questions?**



## Ch. 5 - Structural Similarity

---

### Exercises and Tutorials

- Provided by the instructor during lecture sessions



1<sup>st</sup> project assignment



## Ch. 5 - Structural Similarity

---

### Main References

- Algergawy A.S., Nayak R., and Mesiti M., *XML data clustering: An overview*. ACM Computing Surveys (CSUR), 43(4), 2011
- Marian A.; Abiteboul S. and Mignet L., *Change-Centric Management of Versions in an XML Warehouse. Proceedings of the International Conference on Very Large Data Bases (VLDB)*, pp. 581-590, 2001
- Nguyen B.; Abiteboul S.; Cobena G. and Preda M., *Monitoring XML Data on the Web*. Proceedings of the ACM International Conference on Management of Data (SIGMOD), 2001. pp. 437-448
- Tekli J.; Chbeir R.; and Yetongnon K., *An Overview of XML Similarity: Background, Current Trends and Future Directions*. Elsevier Computer Science Review, 3(3):151-173, 2009
- Tekli et al., *A Novel XML Structure Comparison Framework based on Sub-tree Commonalities and Label Semantics*, Elsevier Journal of Web Semantics (JWS): Science, Services, and Agents on the World Wide Web, 11: 14-40, 2012.
- Tekli J., Chbeir R., *Minimizing user effort in XML grammar matching*. Elsevier Information Sciences Journal, 210: 1-40, 2012
- Tekli J., Damiani E., and Chbeir R., *Using XML-Based Multicasting to Improve Web Service Scalability*. International Journal on Web Service Research, 9(1): 1-29, 2012